

PLANFORGE: Synthesizing Planners from Natural Language with an LLM-Driven Architect–Builder–Runner Pipeline

Anonymous submission

Abstract

Large language models can generate plausible step-by-step plans, but such plans are not necessarily executable or correct. Reliable planning requires translating a natural-language task into a computational procedure that tracks state, enforces constraints, selects valid actions, and searches for a solution. Recent programmatic planning approaches improve reliability by operating over such procedures, but typically assume the underlying *blueprint* (state representation, action structure, and solving strategy) is already defined, often by humans. This is limiting, since natural-language planning requires choosing the representation, the decomposition, and the strategy, not just executing them. PLANFORGE addresses this gap by synthesizing the blueprint that prior methods assume is given. From natural-language task families, PLANFORGE induces a blueprint comprising the state representation, a decomposition into function signatures, an algorithmic solving strategy, and integration logic. An *architect* designs this blueprint, a *builder* compiles it into an executable program using a few validation examples, and a *runner* runs the program on held-out instances under external validation. We evaluate generalization across three benchmark families spanning complementary planning and constraint-satisfaction problems. These include NATURALPLAN (trip, meeting, and calendar planning), PLAN GEN. (an ACPBench-based plan generation benchmark on 15 domains), and STRUCTURESAT (9 structured constraint-satisfaction tasks). Across these benchmarks, PLANFORGE achieves a 100% success rate on NaturalPlan and StructureSAT, while achieving 92.69% accuracy on PlanGen. and outperforming the baselines by a significant margin. Furthermore, PLANFORGE compiles the planner into a standalone executable program that generalizes to new instances without any test-time LLM calls, eliminating API queries and reducing cumulative token costs by up to 15× compared to programmatic baselines.

1 Introduction

Large language models (LLMs) are increasingly used to solve multi-step planning problems expressed in natural language. A common approach asks a model to directly produce a sequence of actions or a final plan from a problem description. While such outputs can appear coherent, fluency alone does not ensure that they satisfy constraints, respect transition

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

dynamics, or remain valid as complexity grows. Prior evaluations have highlighted this gap between plausible reasoning and reliable planning behavior (Valmeekam et al. 2023a). A more robust approach treats LLMs as components within a structured execution pipeline, using them to generate code or programs that can be reliably executed to solve complex planning problems (Katz et al. 2024; Cao et al. 2026; Deik et al. 2026). This shift enables verification: instead of trusting an LLM-generated plan, the system runs a computation and checks whether the result satisfies the domain dynamics and constraints. However, these approaches often assume access to a human-curated planner blueprint, operating over predefined state representations, action definitions and solver strategies. While suitable for classical domains, this assumption limits flexibility when dealing with unseen problems expressed only in natural language.

In settings involving novel, unseen, and previously unstudied problems described purely in natural language, the key challenge is not just solving the problem, but deciding how to represent it: what constitutes a state, which constraints matter, and which computational strategy is appropriate. Unlike domains that admit a predefined canonical representation, natural-language problems must be formalized without a priori knowledge of the state space or constraints. While prior work has translated language into formal planning representations such as PDDL (Lindsay et al. 2017; Miglani and Yorke-Smith 2020; Zhang et al. 2024; Li et al. 2024; Zuo et al. 2025; Oswald et al. 2024) and extended this with LLM-driven construction and refinement (Gestrin, Kuhlmann, and Seipp 2024; Huang, Lipovetzky, and Cohn 2025; Smirnov et al. 2024), these approaches target a fixed formalism and solver. They do not address how to choose the state representation and solving strategy in the first place. Programming-based approaches such as SCOPE (Deik et al. 2026) improve robustness by decomposing tasks into structured workflows for candidate generation and constraint filtering, but they still assume a human-designed blueprint. Similarly, recent work on generalized planning synthesizes reusable policies but assumes access to domain-experts who specify the representation (Chen et al. 2025; Murray et al. 2026). This assumption limits flexibility when dealing with unseen problems expressed only in natural language.

To address this gap (highlighted in Table 1), we introduce PLANFORGE. An *Architect–Builder–Runner* framework

Feature		Direct LLM (CoT) (Yao et al. 2023)	LLM+P (Liu et al. 2023)	ToS (Katz et al. 2024)	AutoToS (Cao et al. 2026)	SCOPE (Deik et al. 2026)	Gen. Planning (Silver et al. 2024)	PLANFORGE (Ours)
Input & Output	Natural language input	✓	✓	×	×	✓	×	✓
	Execution-verified output	×	✓	✓	✓	✓	✓	✓
	Certified no-plan verdict	×	✓	✓	✓	×	×	✓
Blueprint	No human-curated state schema	✓	×	×	×	×	×	✓
	No predefined function signatures	✓	×	×	×	×	×	✓
	No fixed search/solver algorithm	✓	×	×	×	×	×	✓
	Autonomous blueprint synthesis	×	×	×	×	×	×	✓
Generalization	Domain-level reusable planner	×	~	✓	✓	×	✓	✓
	Classical transition system planning	×	✓	✓	✓	×	✓	✓
	Constraint/scheduling planning	~	×	×	×	✓	×	✓
	Scalable to novel / unstructured tasks	✓	×	×	×	×	×	✓

Table 1: **Comparison of representative LLM-based planning paradigms.** Existing frameworks are compared across input/output characteristics, autonomy in blueprint and representation design, and their capability to generalize and reuse solvers. PLANFORGE is the only paradigm that operates directly on natural language input to autonomously synthesize a reusable domain-level planner without human-curated blueprints. (✓ = Supported, × = Not supported, ~ = Partial/limited).

(shown in Figure 1) for synthesizing planners from natural-language domains. Given a domain description and a small set of development examples, PLANFORGE constructs a planner that can be applied to new instances. The *Architect* designs the planner blueprint: it specifies what to extract from language, defines the function signatures, and selects an appropriate solving strategy. The *Builder* turns the blueprint into a procedural program, and the deterministic *Runner* executes the program and verifies its outputs. The result is not just a solution to a single instance, but a reusable planner for a class of problems. This perspective allows the system to adapt its formulation to the task. When a domain admits a clear transition model, it can recover classical search. When constraints dominate, it can instead construct a constraint-based planner. Rather than assuming a human-designed blueprint, PLANFORGE treats the state representation and solving strategy as part of the synthesis problem. We evaluate PLANFORGE on three complementary settings. 1. **NaturalPlan** (Zheng et al. 2024) covering trip planning, meeting scheduling, and calendar reasoning, 2. **Plan Generation** (Plan Gen.) benchmark based on ACPBench (Kokel et al. 2025) that evaluates framework’s ability to work, and 3. **StructureSAT** (Fu et al. 2025), which assesses the framework’s ability to synthesize planners for structured constraint-satisfaction problems.

Our contributions are as follows:

- We formulate the task of synthesizing a *planner blueprint* and generating a reusable *planner* from natural language, without assuming access to a human-designed representation or solving strategy.
- We introduce PLANFORGE, an architect, builder, and runner framework that automates this blueprint design, turns the blueprint into a procedural program using development-set feedback, and executes the program locally.
- We evaluate PLANFORGE across three complementary benchmark families, demonstrating that it achieves a 100% success rate on NaturalPlan and StructureSAT and 92.69% accuracy on Plan Gen without significant increase in cost (See Table 2).

- We show that compiling the planner into a standalone program eliminates test-time API queries, keeping the test-time cost at zero and making it highly cost-effective at scale compared to programmatic baselines that query the model for every test instance.

2 Related Work

We position PLANFORGE in the context of programmatic and neurosymbolic planning. A detailed survey of classical planning grounding, LLM agents, and search-based heuristics is provided in Appendix D.

Programmatic and Neurosymbolic Planning. Recent neurosymbolic frameworks divide labor between neural perception (converting raw inputs to structures) and symbolic reasoning (executing search or checking constraints) (Sheth, Roy, and Gaur 2023). Several approaches translate natural language into formal planning specifications (e.g., PDDL) to run classical solvers (Liu et al. 2023), or compile problems into programmatic workflows that generate and filter candidates. For e.g., SCOPE (Deik et al. 2026). However, these frameworks rely on predefined, human-designed planner blueprints (representations, function signatures, or fixed algorithmic templates). This assumption limits their applicability to novel planning problems described in natural language. In contrast, PLANFORGE automates the end-to-end process of synthesizing planners by dynamically inducing the blueprint.

Search, Heuristics, and Generalized Planning. Interactive LMs generate plans via step-by-step action text, but their outputs are prone to execution errors and unreachable hallucination (Valmeekam et al. 2023b; Kambhampati et al. 2024; Bohnet et al. 2026). To enforce correctness, recent methods prompt LMs to generate search components or heuristic functions (Katz et al. 2024; Corrêa, Pereira, and Seipp 2025; Cao et al. 2026). Other generalized planning works synthesize reusable programs but assume expert-designed schemas (Silver et al. 2024; Chen et al. 2025). PLANFORGE resolves this

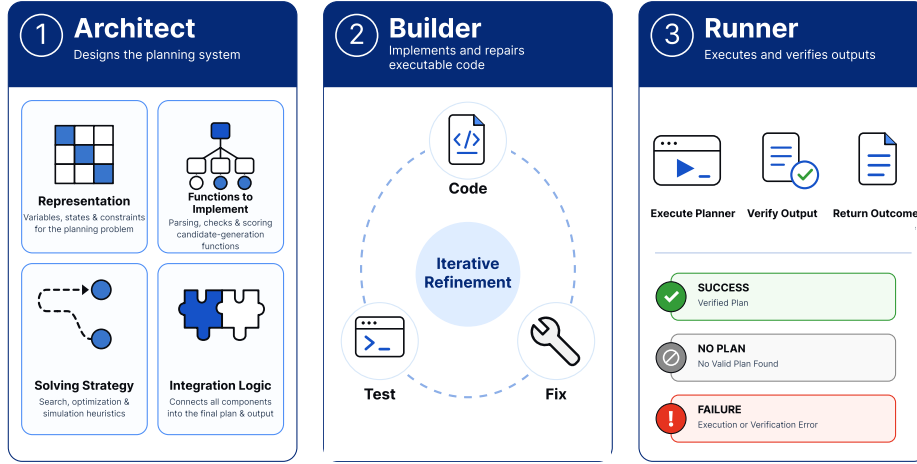


Figure 1: **PLANFORGE Architect, Builder, and Runner pipeline.** Given a natural-language planning problem, the Architect designs the planning system blueprint by specifying the representation, functions to implement, solving strategy, and integration logic. The Builder transforms this blueprint into a procedural program through an iterative refinement loop (Code, Test, Fix). The Runner executes the planner, verifies the output, and returns the outcome (Success, No Plan, or Failure).

	Method	NaturalPlan	Plan Gen	StructureSAT
Perf.	CoT	86.10	0.63	30.37
	SCOPE	96.00	4.80	99.70
	PLANFORGE	100.00	92.69	100.00
Cost	CoT	1.89	4.03	39.96
	SCOPE	3.44	3.66	90.91
	PLANFORGE	3.21	16.77	6.10

Table 2: Comparison of performance and cost of PLANFORGE and with Chain-of-Thought and SCOPE on NaturalPlan, Plan Gen., and StructureSAT. Performance is measured using success rate for NaturalPlan and StructureSAT, and accuracy for Plan Gen.

bottleneck by synthesizing both the search strategy and the underlying representation autonomously.

3 PLANFORGE

We introduce PLANFORGE, an autonomous framework that automates planner synthesis directly from natural-language domain descriptions using a few validation examples. PlanForge is a Architect-Builder-Runner pipeline, where the architect designs a blueprint for a planner, the builder turns the blueprint into a procedural program and the runner executes this program on each problem instance.

3.1 Architect: Design the Planner Blueprint

The architect determines *how* a domain should be solved. Given the domain description d and validation examples V , the architect designs the planner blueprint Φ . This blueprint specifies:

- State representation: the variables, states, and constraints that define the planning problem.

Benchmark	CoT	SCOPE	PLANFORGE
Trip Planning	84.60	100.00	100.00
Meeting Planning	93.20	97.33	100.00
Calendar Scheduling	80.51	90.67	100.00
NaturalPlan (Overall)	86.10	96.00	100.00

Table 3: Comparison of Success Rate (%) on the NaturalPlan benchmark across the evaluated methods.

- Core functions: the procedures to implement, including parsing, validation checks, and candidate-generation function signatures.
- Solving strategy: the underlying algorithmic paradigm, such as search, optimization, or heuristic reasoning.
- Integration logic: the mechanisms that connect all components and produce the final plan.

Crucially, the architect is not restricted to a predefined template, it autonomously determines the computational abstraction suitable for the task, choosing between search, backtracking, constraint satisfaction, or domain-specific heuristics. We present an example blueprint in Appendix B

3.2 Builder: Turn the Blueprint to a Procedural Program

The builder transforms the architect’s blueprint Φ into a procedural program P . The program implements the state representation, functions, solving strategy, and integration logic specified in Φ .

We implement the builder as an LLM coding agent equipped with file-editing and code-execution tools. The agent accesses the model to write, test, and debug the program P in an iterative refinement loop (Code, Test, Fix). In

Domain	CoT	SCOPE	PLANFORGE
Alfworld	0.00	0.00	68.75
Blocksworld	0.00	3.12	100.00
Depot	0.00	3.12	100.00
Ferry	0.00	3.12	100.00
Floortile	0.00	6.25	100.00
Frogs_Jumping	0.00	0.00	100.00
Goldminer	0.00	3.12	93.75
Grid	0.00	15.62	96.88
Grippers	0.00	3.12	100.00
Hanoi	0.00	3.12	34.38
Logistics	3.12	3.12	100.00
Rovers	0.00	3.12	100.00
Satellite	6.25	3.12	96.88
Swap	0.00	3.12	100.00
Visitall	0.00	18.75	100.00
Plan Gen (Overall)	0.63	4.80	92.69

Table 4: Comparison of **Accuracy (%)** on **Plan Gen.**

each iteration, the program state is updated based on feedback collected by executing the current program on the validation set V . If the feedback indicates that the initial blueprint Φ itself is flawed (e.g., due to an incorrect state representation or missing constraint abstractions), this feedback is routed back to the architect to revise and update the blueprint.

3.3 Runner: Execute the Program

The runner serves as the core execution engine of the framework. The runner is completely deterministic and contains no language-model reasoning. Its role is to execute the program P on each problem instance to return the execution outcome o . During test-time, PLANFORGE directly invokes the runner with the determined program P and the test instance.

3.4 Iterative Feedback Loop

A key feature of PLANFORGE is the iterative feedback loop coordinating the architect, builder, and runner during synthesis. After each iteration of program synthesis by the builder, the intermediate program P is executed by the runner on the validation instances $v \in V$. The resulting execution outcome o are processed by a feedback module, which maps the outcome to one of three labels: SUCCESS, indicating that a valid plan was produced and verified; NO PLAN, indicating that no valid plan was found or that the program correctly determined that no feasible plan exists; and FAILURE, indicating an execution or verification error, such as a syntax error, runtime exception, timeout, or invalid output. The execution outcome o and associated feedback f are then routed to either the builder or the architect. This feedback captures syntax errors, runtime exceptions, constraint assertion failures, and reference mismatches.

Feedback is directed to the builder when implementation-level issues must be corrected, and to the architect when the outcome suggests deficiencies in the blueprint Φ , such as an inadequate state representation or solution strategy. This

iterative refinement process continues until the synthesized program successfully solves all validation instances in V or the interaction budget is exhausted.

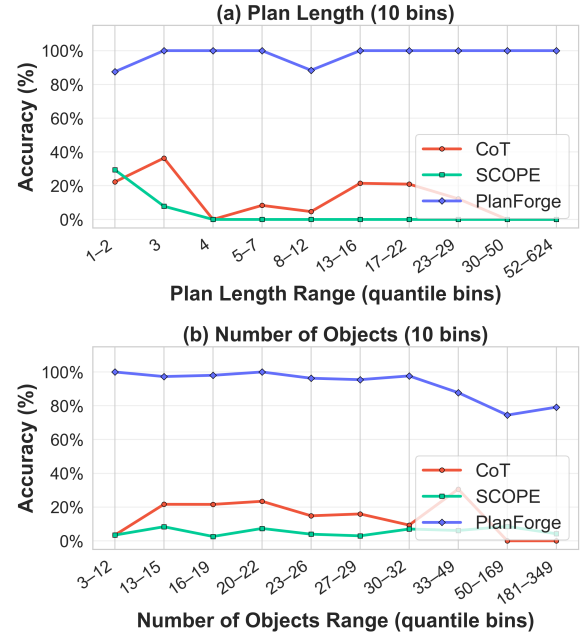


Figure 2: **Plan Gen. results.** Plan-generation accuracy across (a) plan lengths and (b) problem complexity. PLANFORGE achieves higher overall accuracy and remains reliable on medium, long, and very-long plans.

4 Experimental Setup

We evaluate whether PLANFORGE can effectively and efficiently synthesize task-specific planners from natural-language problem descriptions, and assess the ability of the resulting planners to correctly solve held-out test instances.

4.1 Benchmarks

We evaluate PLANFORGE across three benchmark families that progressively increase in complexity and diversity. We begin with **NaturalPlan** (Zheng et al. 2024), which provides planning tasks described entirely in natural language. While these tasks capture realistic planning scenarios, they typically involve relatively small search spaces and short solutions, with plan lengths usually below 25 steps. To evaluate synthesis under larger search spaces and longer horizons, we introduce **Plan Gen.**, an ACPBench-inspired benchmark based on PDDL domains where the number of objects and instances can be systematically controlled, yielding instances with plans exceeding 600 actions. Finally, we consider **StructureSAT** (Fu et al. 2025) to assess whether PLANFORGE can generalize beyond planning and synthesize solvers for structured sequential decision-making and constraint-satisfaction problems. Next we present each of the dataset.

NaturalPlan. Zheng et al. (2024) introduced NaturalPlan, a benchmark comprising of trip planning, meeting planning, and

calendar scheduling tasks with interacting temporal, spatial, and budget constraints. These domains evaluate whether PLANFORGE can synthesize an appropriate constraint-oriented representation and solution procedure directly from natural-language task descriptions, without access to an explicit transition model. Following the approach in SCOPE (Deik et al. 2026), we measure the performance using success rate, defined as the fraction of generated plans that satisfy all task constraints. The dataset provides a golden plan for each instance, in our evaluation we first check if the generated plan has exact-match, in case the generated plan does not have exact-match we use LLM-based judge to verify if the task constraints are satisfied in the generated plan. We randomly sampled 20 examples as a validation set for synthesis and refinement, and evaluate the final synthesized planner on a randomly sampled 150 instances, treated as held-out test set.

Plan Generation Kokel et al. (2025) introduced ACPBench collection that uses PDDL domain and problem descriptions to generate dataset to evaluate atomic planning tasks. Inspired by this work that allows systematic control over number of instances and size of the state space, we generate Plan Generation benchmark, following their pddl to natural language conversion closely using templates. ACPBench originally consisted 11 classical planning domains, Alfworld (Shridhar et al. 2021), and a novel swap domain. Inspired by Shojaei et al. (2025) we further added Tower of Hanoi and Frogs Jumping domains, yielding 15 domains in total. Plan Gen. has long-horizon problem over these domains, with plans exceeding 600 actions. It tests whether a planner synthesized from the domain description continues to generate valid action sequences as problems have increasing plan length, number of objects, and state space size. This benchmark has 45 examples in each domain and we use 9 of those for validation set. Since each of these domains have associated PDDL, we were able to use plan validation (VAL) software (Howey, Long, and Fox 2004) to verify the accuracy of the generated plan. Our results report this VAL-verified plan-generation accuracy for Plan Gen., unless stated otherwise.

StructureSAT. Fu et al. (2025) introduced StructureSAT, a benchmark comprising nine families of structured satisfiability problems spanning diverse combinatorial domains. Unlike the planning benchmarks above, the objective is to synthesize a procedure that determines satisfiability and produces a valid satisfying assignment whenever one exists. StructureSAT provides a complementary evaluation setting that tests whether PLANFORGE can synthesize executable solvers for structured constraint-satisfaction problems rather than action-sequence generators. For this benchmark, we report success rate as the fraction of instances for which the synthesized solver returns a valid satisfying assignment when one exists. Unlike the other two datasets, StructureSAT does not have a natural language description, rather each problem instance is a ‘.cnf’ file. Hence, the domain-description was empty for PLANFORGE, but the LLMs were provided validation set with folder name indicating the family of problem. For our evaluations, we sampled 20 examples for validation set and 150 examples for the held-out test set in each family.

Domain	CoT	SCOPE	PLANFORGE
3-sat	2.67	100.00	100.00
automor	6.67	100.00	100.00
ca	0.00	98.00	100.00
k-clique	22.67	100.00	100.00
k-color	78.67	100.00	100.00
sr	4.00	100.00	100.00
k-domset	93.33	99.33	100.00
k-vercov	62.67	100.00	100.00
ps	2.67	100.00	100.00
StructureSAT (Overall)	30.37	99.70	100.00

Table 5: Comparison of Success Rate (%) of Chain-of-Thought (CoT), SCOPE, and PLANFORGE.

4.2 Experimental Setup

All experiments access language models via public API calls, employing GPT-5 . 4 for architectural reasoning and Codex for the builder. Development and evaluation are strictly separated. During development, PLANFORGE synthesizes and refines the program using feedback from few validation examples.¹ and single harness throughout the main paper. Other open-source models are provided in the appendix. Once development is complete, the synthesized program is frozen and evaluated on held-out test instances that are never exposed during synthesis. Importantly, the final program operates independently of the underlying LLM: at test time, it executes deterministically and requires no LLM calls.

We compare PLANFORGE against two baselines: zero-shot Chain-of-Thought prompting and SCOPE. Because the implementation of SCOPE was not publicly available, we reimplemented the method based on the details provided in the paper and used this implementation for all comparisons.

5 Results and Analysis

We present the aggregated results in Table 2 and analyze the results for each benchmark in turn below.

5.1 NaturalPlan

NaturalPlan does not expose an explicit transition model. Instead, solving an instance requires identifying the relevant decision variables and constraints directly from natural-language descriptions, such as city visitation order and travel dates in trip planning, or availability windows and scheduling constraints in meeting and calendar planning. PLANFORGE addresses this challenge by inducing a constraint-oriented representation and an executable search or constraint-satisfaction procedure. Given a natural-language problem instance, the synthesized planner constructs a structured state representation, generates candidate solutions, and verifies them against the extracted constraints.

Table 3 shows that PLANFORGE achieves a success rate of 100% across all three NaturalPlan domains, outperforming

¹We focus on a single model

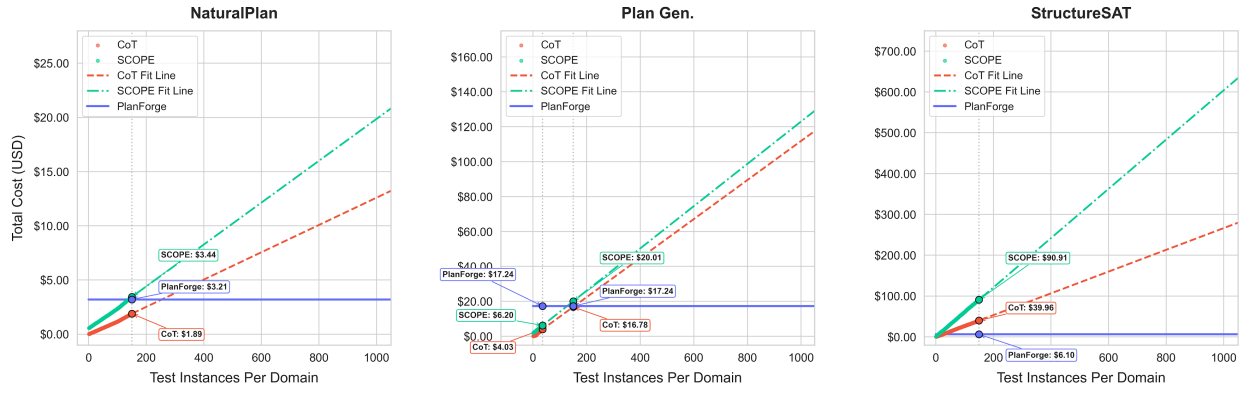


Figure 3: Cumulative API cost (USD) versus the number of evaluated test instances, including development and test-time costs. PlanForge incurs a one-time development cost and requires no test-time API queries, so its total cost remains constant. In contrast, CoT and SCOPE query the model for each instance, causing their costs to increase with evaluation size.

SCOPE (96.0%) and CoT (86.1%). This improvement stems from the verification-centric nature of PLANFORGE: candidate solutions are accepted only after the synthesized procedure checks that all temporal, spatial, and budget constraints are satisfied. In contrast, direct generation methods must satisfy these constraints while generating free-form text. Exact Match (EM) provides an additional measure of solution fidelity. While CoT and SCOPE achieve 0% and 90.89% EM, respectively, under the benchmark’s strict reference-order criterion, PLANFORGE attains 97.55%, indicating that the synthesized procedures not only produce valid solutions but also closely match the benchmark’s canonical solution format.

5.2 Plan Generation

Plan Gen evaluates a complementary challenge: maintaining correct transition dynamics while generating long action sequences. For each domain, PLANFORGE synthesizes a state representation, transition model, and search procedure from the natural-language description, and executes the resulting program to generate a plan. The generated action sequence is then checked by VAL. Thus, a syntactically plausible action sequence receives no credit if an action is unavailable, a precondition is violated, a state transition is incorrect, or the goal is not reached.

Table 4 presents the resulting VAL-verified accuracy across 15 domains. PLANFORGE achieves an overall accuracy of 92.69%, substantially outperforming SCOPE (4.80%) and zero-shot CoT (0.63%). The large performance gap highlights the challenge of generating long, executable plans while maintaining a consistent symbolic state and respecting domain dynamics throughout the planning process. Although the direct-generation baselines occasionally produce locally plausible actions, they rarely generate complete action sequences that remain valid under execution.

Figure 2 provides a more detailed breakdown of performance across plan lengths and problem sizes. Across the plan-length and object-count bins, PLANFORGE remains reliable for medium, long, and very-long plans, including plans with hundreds of actions. Its errors are concentrated in a small number of difficult domains (specifically Hanoi and

Domain	Guided	Autonomous
alfworld	33.30	100.00
blocksworld	100.00	100.00
depot	93.30	100.00
ferry	100.00	100.00
floortile	100.00	100.00
frogs jumping	100.00	100.00
goldminer	80.00	100.00
grid	86.70	100.00
grippers	100.00	100.00
hanoi	100.00	100.00
logistics	93.30	100.00
rovers	100.00	100.00
satellite	86.70	100.00
swap	100.00	100.00
visitall	93.30	100.00
Overall	91.10	100.00

Table 6: Comparison of Guided and Autonomous PLANFORGE on Plan Gen Easy under the short-plan setting (containing 0–2 actions). Results are reported as Accuracy (%).

AlfWorld, see Table 4). These results suggest that accurately capturing and implementing the domain dynamics is the primary challenge, whereas generating long action sequences is comparatively straightforward once a correct planner has been synthesized.

5.3 StructureSAT

StructureSAT isolates a complementary setting in which success requires a valid satisfying assignment rather than a sequential action plan. Here, PLANFORGE induces a constraint-satisfaction formulation and executes the compiled solver to produce an assignment or an UNSAT verdict. Table 5 shows that PLANFORGE achieves a perfect 100.00% success rate, compared with 99.70% for SCOPE and 30.37% for CoT. We

Benchmark	Metric	CoT	SCOPE	PlanForge
Plan Gen.	Plan-existence accuracy	98.89	11.31	98.89
	Plan-generation accuracy	13.78	5.37	92.69
StructureSAT	SAT/UNSAT prediction accuracy	75.26	99.70	100.00
	Success rate	30.37	99.70	100.00

Table 7: Prediction and solution-synthesis performance (%). Predicting whether a solution exists can substantially overestimate the ability to construct a verifiable solution.

additionally studied SAT/UNSAT prediction accuracy (following (Fu et al. 2025)), which measures whether a method correctly predicts the satisfiability status of an instance, regardless of whether it produces a satisfying assignment. Under this metric, CoT achieves 75.26% accuracy; however, its success rate drops to 30.37% when required to construct a valid satisfying assignment. This gap highlights the difference between recognizing that a solution exists and synthesizing an executable procedure that actually computes one. This observation motivates our next experimental setting, where we directly compare plan-existence prediction against executable plan generation to quantify how much performance degrades when methods are required to produce verifiable solutions rather than merely predict their existence.

5.4 Prediction versus Solution Synthesis

Across all settings, predicting that a solution exists is substantially easier than constructing one. A solution-existence prediction requires only a binary decision about whether a valid solution can be found, whereas solution generation requires producing a witness that can be independently verified against the task constraints. In Plan Gen., this witness is an action sequence checked by VAL. In NaturalPlan, it is a schedule satisfying the specified constraints. In StructureSAT, it is an assignment accepted by the symbolic verifier. PLANFORGE explicitly addresses this distinction by executing the synthesized program and categorizes the outcome to either: a verified solution, a correct no-solution verdict, or a failure. This execution boundary prevents fluent but invalid output from being counted as a successful plan.

Table 7 makes this gap explicit. On Plan Gen., CoT matches PLANFORGE on plan-existence accuracy (98.89%) but achieves only 13.78% plan-generation accuracy, compared with 92.69% for PLANFORGE. The same pattern appears in StructureSAT, where CoT’s 75.26% SAT/UNSAT prediction accuracy falls to a 30.37% assignment success rate. Across these settings, execution-verified synthesis is the more discriminating measure of planning capability.

5.5 Planner Synthesis Cost

Table 2 presents the aggregated cost of planner synthesis using PLANFORGE as compared to the CoT and SCOPE approaches (a detailed breakdown is in Appendix C.3). PLANFORGE incurs an upfront synthesis cost to design and implement a task-specific planner. Once synthesized, however, the resulting program can be reused across arbitrarily many instances without additional LLM calls. In contrast, CoT and SCOPE invoke a model for every new instance, causing inference

costs to grow linearly with workload. Figure 3 illustrates this amortization effect: although planner synthesis incurs a one-time cost, the compiled planner can subsequently solve new instances through local execution alone. Consequently, the appropriate comparison is not merely per-instance inference cost, but the trade-off between a one-time planner compilation cost and repeated online generation.

5.6 Evaluating Autonomous Blueprint Induction

The primary system configuration is Autonomous PLANFORGE, in which the Architect independently selects the problem representation and solution procedure for each domain. To assess the importance of this capability, we introduce Guided PLANFORGE, an ablation that replaces autonomous blueprint induction with a manually specified blueprint.

Inspired by Katz et al. (2024) and Cao et al. (2026), we used PDDL representation, fixed `is_goal` and `successor` functions, and a predefined greedy best-first search strategy for Plan Gen. For NaturalPlan, we replace the PDDL representation with a JSON-based state representation while keeping the remainder of the blueprint unchanged.

Table 6 compares the two variants on NaturalPlan and Plan Gen. Easy, a short-horizon setting whose reference plans contain at most two actions. Both variants achieve 100% success rate on NaturalPlan, but Autonomous PLANFORGE reaches 100% accuracy on Plan Gen. Easy versus 91.10% for Guided PLANFORGE. The gap is largest in *Alfworld* and *Goldminer* domains, where the PDDL representation introduced grounding mismatches an inefficient search behavior. These results indicate that allowing the Architect to adapt both the representation and planning procedure to the domain is a key contributor to overall performance.

6 Conclusion and Discussion

We introduce PLANFORGE, an Architect, Builder, and Runner framework that automates planner synthesis by inducing state representations, function decompositions, and solving strategies. Across NaturalPlan, Plan Gen., and StructureSAT, it consistently outperformed direct-generation baselines on natural-language planning, long-horizon classical planning, and structured constraint-satisfaction problems. The results show that executable planners can reliably construct verifiable solutions where language-model generation often recognizes solutions but struggles to produce valid witnesses. The synthesized planners also generalize to unseen instances without test-time LLM calls, amortizing the one-time synthesis cost across repeated deployment. Together, these findings suggest that compiling reusable planners from natural-language specifications is a promising alternative to instance-by-instance generation. More broadly, language models can act as planner designers rather than online decision makers, synthesizing executable procedures that are independently verified, reused, and scaled to increasingly complex reasoning tasks.

Limitations. PLANFORGE relies on LLM, misinterpreting task descriptions during blueprint generation can lead to incorrect formalizations. Additionally, validation-set feedback can bias synthesized programs toward benchmark-specific plan styles.

References

- Bohnet, B.; Mozer, M. C.; Swersky, K.; Cunningham, W.; Parisi, A.; Kenealy, K.; and Fiedel, N. 2026. Analysis of Optimality of Large Language Models on Planning Problems. *arXiv:2604.02910*.
- Cao, D.; Katz, M.; Kokel, H.; Srinivas, K.; and Sohrabi, S. 2026. Automating Thought of Search: A Journey Towards Soundness and Completeness. In *ICAPS*.
- Chen, D. Z.; Zenn, J.; Cinquin, T.; and McIlraith, S. A. 2025. Language Models For Generalised PDDL Planning: Synthesising Sound and Programmatic Policies. *CoRR*, abs/2508.18507.
- Corrêa, A. B.; Pereira, A. G.; and Seipp, J. 2025. Classical Planning with LLM-Generated Heuristics: Challenging the State of the Art with Python Code. In *NeurIPS*, volume 38, 42070–42108. Curran Associates, Inc.
- Deik, D. G. X.; Long, Q.; Liu, Z.; Chen, N. F.; and Wang, W. 2026. Programming over Thinking: Efficient and Robust Multi-Constraint Planning. *arXiv:2601.09097*.
- Fu, Y.; Tompkins, A.; Song, Y.; and Pagnucco, M. 2025. Structure based SAT dataset for analysing GNN generalisation. In *AISTATS*, volume 258 of *Proceedings of Machine Learning Research*, 4834–4842. PMLR.
- Gestrin, E.; Kuhlmann, M.; and Seipp, J. 2024. NL2Plan: Robust LLM-Driven Planning from Minimal Text Descriptions. *CoRR*, abs/2405.04215.
- Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In *ICTAI 2004*, 294–301. IEEE Computer Society.
- Huang, S.; Lipovetzky, N.; and Cohn, T. 2025. Planning in the Dark: LLM-Symbolic Planning Pipeline Without Experts. In *AAAI*, 26542–26550.
- Kambhampati, S.; Valmeekam, K.; Guan, L.; Verma, M.; Stechly, K.; Bhambri, S.; Saldyt, L.; and Murthy, A. 2024. Position: LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks. In *ICML*. OpenReview.net.
- Katz, M.; Kokel, H.; Srinivas, K.; and Sohrabi, S. 2024. Thought of Search: Planning with Language Models Through The Lens of Efficiency. In *NeurIPS*.
- Kokel, H.; Katz, M.; Srinivas, K.; and Sohrabi, S. 2025. ACPBench: Reasoning About Action, Change, and Planning. In *AAAI*, 26559–26568. AAAI Press.
- Li, R.; Cui, L.; Lin, S.; and Haslum, P. 2024. NaRuto: Automatically Acquiring Planning Models from Narrative Texts. In *AAAI*, 20194–20202. AAAI Press.
- Lindsay, A.; Read, J.; Ferreira, J. F.; Hayton, T.; Porteous, J.; and Gregory, P. 2017. Framer: Planning Models from Natural Language Action Descriptions. In *ICAPS*, 434–442. AAAI Press.
- Liu, B.; Jiang, Y.; Zhang, X.; Liu, Q.; Zhang, S.; Biswas, J.; and Stone, P. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. *CoRR*, abs/2304.11477.
- Miglani, S.; and Yorke-Smith, N. 2020. NLtoPDDL: One-Shot Learning of PDDL Models from Natural Language Process Manuals. In *KEPS at ICAPS*.
- Murray, A.; Dervovic, D.; Pozanco, A.; and Cashmore, M. 2026. GenePlan: Evolving Better Generalized PDDL Plans using Large Language Models. *CoRR*, abs/2603.09481.
- Oswald, J. T.; Srinivas, K.; Kokel, H.; Lee, J.; Katz, M.; and Sohrabi, S. 2024. Large Language Models as Planning Domain Generators. In *ICAPS*, 423–431.
- Sheth, A. P.; Roy, K.; and Gaur, M. 2023. Neurosymbolic Artificial Intelligence (Why, What, and How). *IEEE Intell. Syst.*, 38(3): 56–62.
- Shojaee, P.; Mirzadeh, I.; Alizadeh-Vahid, K.; Horton, M.; Bengio, S.; and Farajtabar, M. 2025. The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity. In *NeurIPS*.
- Shridhar, M.; Yuan, X.; Côté, M.; Bisk, Y.; Trischler, A.; and Hausknecht, M. J. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *ICLR*. OpenReview.net.
- Silver, T.; Dan, S.; Srinivas, K.; Tenenbaum, J. B.; Kaelbling, L. P.; and Katz, M. 2024. Generalized Planning in PDDL Domains with Pretrained Large Language Models. In *AAAI*, 20256–20264. AAAI Press.
- Smirnov, P.; Joubin, F.; Ceravola, A.; and Gienger, M. 2024. Generating consistent PDDL domains with Large Language Models. *CoRR*, abs/2404.07751.
- Valmeekam, K.; Marquez, M.; Hernandez, A. O.; Sreedharan, S.; and Kambhampati, S. 2023a. PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. In *NeurIPS*.
- Valmeekam, K.; Marquez, M.; Sreedharan, S.; and Kambhampati, S. 2023b. On the Planning Abilities of Large Language Models - A Critical Investigation. In *NeurIPS*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*. OpenReview.net.
- Zhang, T.; Zhang, L.; Hou, Z.; Wang, Z.; Gu, Y.; Clark, P.; Callison-Burch, C.; and Tandon, N. 2024. PROC2PDDL: Open-Domain Planning Representations from Texts. *CoRR*, abs/2403.00092.
- Zheng, H. S.; Mishra, S.; Zhang, H.; Chen, X.; Chen, M.; Nova, A.; Hou, L.; Cheng, H.-T.; Le, Q. V.; Chi, E. H.; and Zhou, D. 2024. NATURAL PLAN: Benchmarking LLMs on Natural Language Planning. *arXiv:2406.04520*.
- Zuo, M.; Velez, F. P.; Li, X.; Littman, M.; and Bach, S. H. 2025. Planetarium: A Rigorous Benchmark for Translating Text to Structured Planning Languages. In *NAACL (Long Papers)*, 11223–11240. Association for Computational Linguistics.

A Implementation and Reproducibility Details

This section will document the Architect–Builder–Runner prompts, the distinction between guided and autonomous implementation details, and the development and validation protocol. It will provide the implementation details needed to understand and reproduce the execution pipeline without including full prompt transcripts.

A.1 Guided and Autonomous PLANFORGE

The Architect–Builder–Runner decomposition lets us compare two variants of PLANFORGE. In the *guided* (restricted) variant, part of the architecture is fixed in advance. For example, the state representation, function signatures, and search procedure may be specified externally, and the Builder is only asked to implement missing components such as state extraction, successor generation, and goal testing. This corresponds to a predefined blueprint design:

$$\Phi^{\text{guided}} = (R^*, F^*, S^*, I^*), \quad (1)$$

where the design components are prescribed rather than chosen by the model.

In the *autonomous* variant, the LLM acts as the Architect and chooses the full design:

$$\Phi^{\text{auto}} = A(d, E), \quad (2)$$

with no fixed state representation, no fixed function signatures, and no prescribed search algorithm. This is the primary form of PLANFORGE studied in this paper. The guided variant serves as a controlled comparison that isolates the effect of delegating architectural choices to the LLM.

This distinction is important: guided PLANFORGE asks the LLM to implement a human-designed blueprint, whereas autonomous PLANFORGE asks the LLM to design the blueprint and implement the program itself. In both cases, correctness is judged only by the Runner’s deterministic verification, not by the plausibility of the generated text.

A.2 Architect–Builder–Runner Implementation Details

To better understand the contribution of the proposed Architect–Builder–Runner (ABR) framework, we implement two variants of the pipeline. The primary system presented throughout the paper is **PLANFORGE-Autonomous ABR**, which is the proposed method and is used for all main experimental results. In this setting, the LLM autonomously synthesizes the complete computational blueprint of the planner, including the state representation, computational functions, and solving strategy, directly from the natural-language domain description.

In addition, we implement a secondary variant, referred to as **PLANFORGE-Guided ABR**, solely as an *ablation study*. Rather than allowing the Architect to autonomously design the planner, this variant replaces the Architect with manually specified planning abstractions provided by human experts. The Builder and Runner remain unchanged, allowing us to isolate the impact of autonomous planner synthesis while keeping the remaining components identical. Guided ABR is therefore not proposed as a competing planning system, but rather serves as a controlled baseline for analyzing the contribution of the autonomous Architect.

The following sections describe the implementation details of both variants, and here in Appendix we refer the Plan Gen. and Plan Gen Easy as **ACPBench** which are subsets of it.

PLANFORGE-Guided ABR detailed explanation In the Guided ABR variant, the LLM’s role is strictly bounded to ensure classical search guarantees. The pipeline is partitioned into three phases:

Architect Phase (Static/Predefined): Human experts define a rigid planning framework. This includes specifying the exact state representation schema (e.g., for trip planning: current city, visited cities, remaining days), defining the integration contracts, and providing ready-made, deterministic classical search algorithms (such as BFS or DFS). No LLM generation occurs in this phase.

Builder Phase (LLM Agent): The LLM intelligence is heavily restricted to problem-specific logic implementation. First, an NL2State agent extracts structured symbolic components from the natural language prompt, enforcing strict schema consistency. Next, the Builder writes Python functions for the initial state configuration, the successor function (transition dynamics), and the goal-test predicate. Crucially, the Builder operates within a test-driven refinement loop: it generates the functions, executes auto-generated unit tests, collects runtime errors, and patches its code iteratively until all semantic and structural tests pass.

Runner Phase (Execution): A deterministic search runner connects the Architect’s search algorithm to the Builder’s generated transition functions. The Runner executes classical search over the induced model. If the search reaches a valid goal state, it returns the generated plan; if it exhausts the reachable state space, it returns an explicit $\emptyset$ (no-plan) verdict. Because the plan is generated by search over an executable model, any returned plan inherently respects the domain constraints.

PLANFORGE-Guided ABR Prompt for ACPBench The Architect and Runner are predefined in the Guided variant, this document serves as the structural contract that the pipeline strictly follows, leaving the dynamic generation entirely to the Builder prompt to build the Function decompositions.

Guided Architect and Runner via Architect–Builder–Runner (ABR) for ACPBench

Role Overview In the Guided variant of PLANFORGE applied to ACPBench, the Architect and Runner roles are completely deterministic and predefined by human experts. No LLM intelligence is used to design the system structure or execute the final search. The framework dictates a rigid integration contract that the Builder must target.

1. Architect Phase (Static System

Design) The Architect defines the fixed abstract representations and integration contracts for the domain. For any given ACPBench domain, the framework enforces the following:

- **State Schema:** The state must be represented as a deterministic symbolic data structure (e.g., a set of parsed relational tuples or a frozen dictionary of propositions).
- **Abstract Methods:** The system requires exactly two domain-specific functional components to be implemented by the Builder:

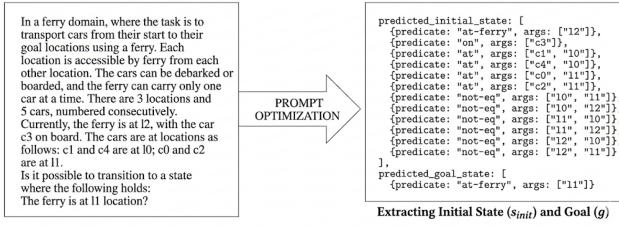


Figure 4: NL2state example

Guided Architect and Runner via Architect–Builder–Runner (ABR) for ACPBench (continued)

- `is_goal(state)`: A boolean function determining if the current state satisfies the problem's goal conditions.
- `get_successors(state)`: A transition function returning all valid (action_name, next_state) pairs reachable from the current state.
- **Search Library**: The Architect mandates the use of a predefined classical search algorithm (such as Breadth-First Search) provided natively in the repository. The LLM is strictly prohibited from altering or generating the search logic.

2. Runner Phase (Deterministic Execution)

The Runner acts as the execution pipeline. It operates as a black-box evaluator with zero LLM intervention.

1. **Initialization**: It imports the predefined search algorithm alongside the `is_goal` and `get_successors` functions generated by the Builder.
2. **Execution**: It initiates the classical search over the induced symbolic model starting from the parsed initial state.
3. **Evaluation**:
 - If the search discovers a valid path to a goal state, the Runner returns the generated action sequence.
 - If the search exhausts the reachable state space without satisfying the goal predicate, the Runner explicitly returns a $\emptyset$ (no-plan) verdict.

The state schema is extracted as shown in the NL2State diagram (see 4). The initial state, goal conditions, and domain descriptions are provided as a planner blueprint to the Builder agent, which leverages the development dataset and the function signatures defined by the Architect to produce the procedural programs. Finally, the Runner executes these programs to output either a candidate plan, a no-plan

verdict, or a failure.

Guided Builder via Architect–Builder–Runner (ABR) for ACPBench

Role

You are an autonomous **CLI coding agent** operating on the repository at {REPO_ROOT} with permission to run shell commands and modify files. Your mission is to make *minimal, correct* changes so that executing `python test_{DOMAIN}.py` finishes with **ALL TESTS PASSED**.

This markdown is meant to be **filled and sent** to your CLI agent (Gemini/Codex/Qwen). The CLI agent will do *all* the execution: read files, run tests, patch code, and write success flags.

- **DOMAIN**: {DOMAIN}
- **DOMAIN DESCRIPTION**: {DOMAIN_DESCRIPTION}
- **REPO ROOT**: {REPO_ROOT}
- **GOAL**: {GOAL}
- **TEST COMMAND**: `python test_{DOMAIN}.py`
- **STATE (is_goal)**: {IS_GOAL_PATH}
- **STATE (succ)**: {SUCC_PATH}

Hard Rules (must follow exactly)

1. In the code that you write **Cannot and will Never** read, import, parse, or rely on the contents of any test files or domain test JSONs inside your implementation code. Forbidden artifacts include (but are not limited to):
 - All `test_*.py`
 - `goal_tests.json`
 - `succ_tests.json`
 - any `*_tests.json`
2. You **may** execute tests via shell (`python test_{DOMAIN}.py`) to observe results and guide your edits.
3. Keep edits surgical and idiomatic; avoid gratuitous refactors.
4. Do not write to or modify any of the forbidden artifacts listed above.
5. Do not look into any other folder other than your domain {DOMAIN} folder.
6. the `succ.py` and `is_goal.py` must be inside the domain folder.

What you should do (loop until green)

1. Inspect current repository code (exclude reading the forbidden artifacts).

Guided Builder via Architect–Builder–Runner (ABR) for ACPBench (continued)

2. Propose and APPLY edits directly to files in {REPO_ROOT}.
3. Run `python test_{DOMAIN}.py`.
4. Use the output to decide next edits, then repeat from step 2 until **ALL TESTS PASSED**.

Completion & Reporting

- When all tests pass, write two JSON files:
 - {IS_GOAL_PATH} with at least:

```
1 {"is_goal": true, "ts": <unix_time>, "notes": "all tests passed"}
```
 - {SUCC_PATH} with at least:

```
1 {"succ": true, "ts": <unix_time>, "notes": "all tests passed"}
```
- If you conclude you cannot make tests pass in reasonable steps, write the same files with false values and include a short reason in "notes".

Extra Guidance

- Never copy/paste assertions or logic from the tests into implementation.
- Prefer small, targeted patches aligned with current project style.
- Document non-obvious decisions in concise code comments only when necessary.

PLANFORGE-Guided ABR Prompt for NaturalPlan & StructureSAT The Architect and Runner are predefined in the Guided variant. This document serves as the structural contract that the pipeline strictly follows, leaving the dynamic code generation entirely to the Builder coding agent.

Guided Architect and Runner via Architect–Builder–Runner (ABR) for NaturalPlan

OVERALL SYSTEM (MENTAL MODEL) You are still doing PLANFORGE, but reorganized:

- Architect = defines the planning framework (static, reusable)
- Runner = executes search + evaluates

Pipeline becomes: Natural Language → (Builder) → State + Functions → (Runner) → Search → Plan / No Plan.

STEP 1: ARCHITECT (YOU DEFINE THE RULES)

This is static code (no LLM here).

- **state_schema.py**: Define Trip Planning State Representation. What goes inside: Cities visited so far, Current city, Remaining cities, Day

Guided Architect and Runner via Architect–Builder–Runner (ABR) for NaturalPlan (continued)

index, Constraints (if any). Output of this file: A strict schema the Builder MUST follow.

- **abstract_planner.py**: Define abstract methods (function signatures only): `get_initial_state()`, `get_goal_test(state)`, `get_successors(state)`. Initial state: 1. In trip_planning dataset, initial state he haven't visited any city. 2. In Meeting_Planning dataset, initial state is the first constraint given in the constraints key of json files. 3. In calendar_scheduling dataset, initial state is empty means the time slot checking will start from the working hours of that day. No implementation, just signatures.
- **search_library.py**: Provide ready-made search algorithms: BFS, DFS, A* (optional). Each expects: `initial_state`, `successor_fn`, `goal_test_fn`. Important: Do NOT generate these via LLM. Keep deterministic.
- **integration_contract.py**: Define Expected input/output formats between Builder → Runner and Runner → Search. Example: Builder must return: `{"initial_state": ..., "goal_test": function, "successor_fn": function}`.

STEP 3: RUNNER (EXECUTION PIPELINE)

- **executor.py**: Takes Builder output and Loads: `initial_state`, `successor_fn`, `goal_test`.
- **search_runner.py**: Runs: `plan = search_algorithm(initial_state, successor_fn, goal_test)`. If plan found → return plan, exhausted → return $\perp$. This gives Plan Existence.
- **evaluation.py**: Compare: Generated plan vs golden_plan. Metrics: Exact match, Constraint satisfaction, Plan validity (execution check).

STEP 5 & 6: HOW PLAN EXISTENCE AND GENERATION WORKS

You don't need extra logic. Search naturally gives: Found → Plan exists. Exhausted → No plan. That's your $\perp$ (no-plan) from PLANFORGE. Search returns: [a1, a2, a3, ...]. Convert to: Day 1: City A, Day 2: City B...

The state schema is extracted as shown in the NL2State diagram. The initial states and constraints are given to the Builder coding agent, which makes use of the development dataset and the function signatures defined by the Architect to produce procedural programs.

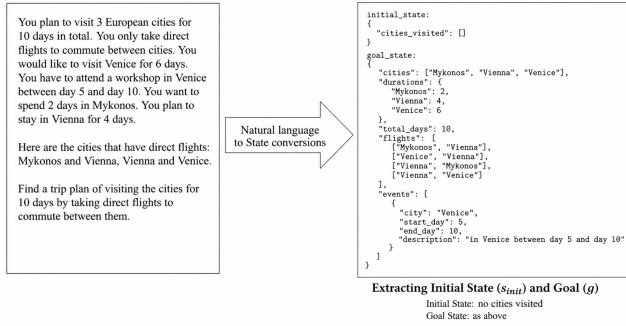


Figure 5: NL2state example for NaturalPlan

The Runner is then executed to provide the candidate plan or no-plan.

Guided Builder via Architect–Builder–Runner (ABR) for NaturalPlan

STEP 2: BUILDER (LLM / CODING AGENT PART) This is where your PLANFORGE intelligence lives.

1. **nl2state_agent.py**: Input: prompt_0shot from dataset. Output: Parsed structured data: cities, durations, constraints. This replaces PLANFORGE NL2State. You MUST enforce schema validation and consistency checks.
2. **function_generator.py**: This generates: 1. Initial State (start city, remaining cities list). 2. Goal Test (all cities visited? total duration matches?). 3. Successor Function (Defines valid transitions: From city A $\rightarrow$ go to city B IF B not visited).
3. **test_generator.py**: Create unit tests automatically. Examples: Cannot visit same city twice, Must visit exactly N cities, Duration constraints respected. This is your PLANFORGE test grounding loop.
4. **refinement_loop.py**: This is your core PLANFORGE loop:


```

1 repeat:
2     generate functions
3     run tests
4     collect errors
5     fix functions
6 until pass or budget exhausted

```

This ensures correctness BEFORE search.

STEP 4: MAIN PIPELINE Flow:

```

1 for each example in trip_planning:
2     # BUILDER
3     structured_data = nl2state_agent()
4     functions = refinement_loop()

```

Guided Builder via Architect–Builder–Runner (ABR) for NaturalPlan (continued)

```

5     # RUNNER
6     result = search_runner()
7     # EVALUATION
8     evaluate(result, golden_plan)
9     save outputs

```

STEP 7: GENERALIZE TO OTHER DATASETS

Once Trip Planning works.

- **Meeting Planning.** State = current location + visited people. Successor = travel + meeting. Constraints = time + distance matrix.
- **Calendar Scheduling.** State = assigned meetings per day. Successor = assign meeting slot. Goal = all meetings scheduled.

Only Builder changes. Architect + Runner stay SAME.

PLANFORGE-Autonomous ABR detailed explanation In the Autonomous ABR variant, the LLM is given maximal freedom to invent the planning system from the ground up. Rather than relying on a human-defined framework, the LLM acts as the lead designer, and the pipeline is structured to execute its vision.

Architect Phase (LLM Agent). Starting with zero prior domain knowledge (given only the raw, natural-language problem description), the architect designs the entire solver system. It has absolute control to determine the state representation schema, decide on the computational solving strategy (e.g., selecting graph search, constraint programming, or custom heuristics), define the necessary abstract function signatures (such as parsers, transition dynamics, and goal checkers), and dictate how these components integrate. The architect outputs a complete, structural specification document.

Builder Phase (LLM Coding Agent). The builder acts as the software engineer, strictly implementing the architect's specification. It writes the Python code to translate natural language into the architect's chosen state representation and implements all the requested functional logic. Crucially, the builder operates inside an agentic, test-driven debugging loop, in which it executes its generated code against a small validation set of instances, inspects runtime errors (such as Python tracebacks or semantic logic failures), and iteratively rewrites the functions until the program behaves correctly and flawlessly handles the validation constraints.

Runner Phase (Execution). The runner is a purely deterministic execution engine with no LLM reasoning involved. It simply takes the builder's verified procedural program, which now inherently contains the search algorithm selected by the architect and the logic written by the builder, and executes it across the remaining test instances. It outputs a generated plan if a valid sequence is found, or returns an explicit $\emptyset$ (no-plan) verdict if the programmatic search space is exhausted. Because the final output is the result of deterministic code execution rather than textual generation, the plans are automatically guaranteed to respect the logic programmed by the builder.

PLANFORGE-Autonomous ABR Prompt for NaturalPlan & StructureSAT For the NaturalPlan dataset (trip planning, meeting planning, calendar scheduling), the following specification document is passed verbatim to the LLM coding agent as the Architect prompt. The agent is free to choose any state representation, any solving strategy, and any computational mechanism, subject

only to the executability and verifiability constraints stated in the document.

Autonomous Planning via Architect–Builder–Runner (ABR)

Core Idea

In this methodology,

We allow an LLM-based agent to:

- Design the planning abstraction
- Choose how to represent the problem
- Decide how planning should be solved

The system only enforces one requirement:

The final solution must be
executable and verifiable

System Overview

The system consists of three roles:

1. Architect (LLM Agent)

The Architect is responsible for designing a planning system for a given natural language problem.

It decides:

- How to represent the problem (state, variables, structures, or any abstraction)
- What functions are required (e.g., transitions, scoring, constraints, or alternatives)
- What solving strategy to use (search, simulation, optimization, heuristic reasoning, etc.)
- How all components interact to produce a plan

The Architect outputs a **complete specification** of a planning system.

2. Builder (LLM Coding Agent)

The Builder implements the system designed by the Architect.

It:

- Translates natural language input into structured representations (if required)
- Implements all functions defined by the Architect
- Generates the procedural program

The Builder is NOT restricted to:

- Classical planning
- Explicit state transitions
- Goal predicates

It can implement any computational mechanism that produces a plan.

3. Runner (Execution Engine)

The Runner executes the system produced by the Builder.

It:

- Loads the generated code
- Runs the planning procedure
- Produces:
 - A candidate plan, or

Autonomous Planning via Architect–Builder–Runner (ABR) (continued)

- A failure signal (no plan found)

The Runner treats the system as a black box.

Key Constraint: Executability and Verification

Even though the Architect has full freedom, the system must satisfy:

1. Executability

- All generated components must run without errors

2. Verifiability

- The produced plan must be checkable against the problem constraints

3. Deterministic Outcome

- Running the same system should produce consistent results

Planning Workflow

For each input example:

Step 1: Architect Phase

Input:

- Natural language problem (from dataset)

Output:

- A structured system design describing:
 - Representation
 - Functions
 - Solving strategy
 - Integration logic

Step 2: Builder Phase

Input:

- Architect specification
- Natural language problem

Process:

- Generate code implementing the system
- Optionally create internal validation checks

Output:

- Procedural program

Step 3: Execution Phase (Runner)

- Execute the generated system
- Obtain:
 - Plan (sequence / structure)
 - OR failure

Step 4: Verification Phase

- Check whether:
 - Plan satisfies constraints
 - Plan matches required structure
 - Plan is logically consistent
- If verification fails:
 - Return failure OR trigger regeneration

Output Definition

```
1 {
2   "status": "SUCCESS" | "FAILURE",
3   "plan": <generated_plan_or_null>
4 }
```

Autonomous Planning via Architect–Builder–Runner (ABR) (continued)

Plan Existence

- If a valid plan is produced → SUCCESS
- If no valid plan is found → FAILURE

No separate logic is required.

Dataset Usage

This framework is applied to:

1. Trip Planning
2. Meeting Planning
3. Calendar Scheduling

Each instance provides:

- Natural language prompt
- Constraints
- Ground truth plan ("golden_plan") for evaluation only

The system must operate **without relying on ground truth during generation**.

Evaluation

After execution:

- Compare generated plan with ground truth ("golden_plan")
- Check:
 - Validity
 - Constraint satisfaction
 - Structural correctness

Metrics:

- Exact match accuracy
- Valid plan rate
- Failure detection accuracy

Important Notes

- Do NOT hardcode planning logic
- Do NOT assume classical planning
- Do NOT enforce state-transition structure
- Allow the agent to invent representations and strategies

However:

- Ensure outputs are executable
- Ensure results are verifiable

Expected Behavior of the Agent

The agent should:

- Explore multiple possible representations
- Choose efficient solving strategies
- Adapt methods across different problem types
- Improve through iterative refinement if needed

Summary

This methodology transforms planning from:

"LLM generates plans"

to:

"LLM designs and builds a system that generates plans"

The correctness of the solution is ensured not by the LLM's text

Autonomous Planning via Architect–Builder–Runner (ABR) (continued)

generation, but by executing and verifying the system it creates.

PLANFORGE-Autonomous ABR Prompt for ACPBench

For ACPBench, the following specification document is passed verbatim to the LLM coding agent as the Architect prompt. The agent is free to choose any state representation, any solving strategy, and any computational mechanism, subject only to the executability and verifiability constraints stated in the document. The corresponding dataset instances and domain validator are provided alongside the methodology document below. The Builder validates its implementation on 10 held-out instances before the Runner evaluates on the 15 original ACPBench test instances.

Architect–Builder–Runner (ABR)

Core Idea

An LLM-based system that **designs, builds, and executes** a planning system (not plans directly).

Each domain starts from scratch. No prior knowledge is carried over between domains.

Roles

Architect (LLM Agent)

The Architect has **full control**. Given only the natural language problem, it decides:

- **State representation:** how to represent the problem (what data structures, what variables)
- **Algorithm:** which solving approach to use (picks from a library, does not implement it)
- **Abstract methods:** defines what functions are needed and their signatures
- **Integration:** how the functions plug into the chosen algorithm to produce a result

The Architect outputs a complete specification. It receives no answer, no ground truth, no PDDL, only the raw problem description.

Builder (LLM Coding Agent)

The Builder implements exactly what the Architect specified. It:

- **Translates** natural language input into the state representation the Architect chose
- **Implements** all abstract methods the Architect defined

The Builder does not decide the approach. It only writes code to fulfill the Architect's design.

Runner (Execution Pipeline)

The Runner is a deterministic executor. It:

Architect-Builder-Runner (ABR) (continued)

- Uses the **algorithm** the Architect selected
- With the **implemented methods** the Builder wrote
- On the **state representation** the Builder populated
- To find the **result** (plan if one exists, or no-plan)

The Runner does not reason. It runs code and returns output.

Pipeline: Per Domain

Each domain is processed **independently from scratch**:

1. **Architect** receives one problem instance (context + inputs only). Outputs the full system specification.
2. **Builder** implements the specification as executable Python. Validates on 10 instances. Revises if needed.
3. **Runner** executes on 15 test instances. Collects results.

Evaluation

Two metrics, computed over the 15 test instances per domain:

Plan Existence (%): Does the system correctly determine whether a valid plan exists?

Compare the system's yes/no output against the ground-truth answer.

Plan Generation (%): Does the system produce the correct plan?

Compare the system's generated plan steps against the ground-truth plan.

Directory Structure

For each domain, the code should be organized modularly:

```
1 {domain}_ABR/
2 |-- architect/    # State schema,
    abstract methods, search
    selection
3 |-- builder/      # NL-to-State
    translation, method
    implementations
4 |-- runner/       # Execution
    pipeline and evaluation logic
5 |-- outputs/      # Generated plans
    and result logs
6 `-- main.py       # Entry point to
    orchestrate ABR pipeline
```

Final evaluation across all 15 domains will yield summary.csv with Plan Existence % and Plan Generation % metrics.

the current program P on the development set V , summarizes the observed failures, and routes those failures to the agent that owns the corresponding decision. After the development gate is passed, the synthesized program is frozen. The held-out or full-domain evaluation then runs the frozen program without further changes to the architect specification Φ or the builder implementation.

Agent responsibilities. The three agents have different ownership boundaries. The architect owns the planning abstraction, which includes the state representation, transition model, search policy, tie-breaking rule, verifier semantics, and output contract. The builder owns the executable realization of that abstraction, namely parsing, solver code, rendering, normalization, and repair of implementation bugs. The runner owns black-box execution and measurement. It loads the current builder code, runs it on each development instance, captures the generated plan and status, checks semantic validity, and compares against the development reference solution when exact match is used as a development signal.

Feedback packet. For each development iteration, the runner returns a feedback packet rather than a single scalar score. The packet identifies the failing instance ids and separates the failure mode into actionable categories.

- **Execution feedback:** syntax errors, import failures, exceptions, unsupported input formats, or crashes while constructing or running P .
- **Semantic feedback:** plans that execute but violate domain constraints, such as unavailable calendar intervals, infeasible travel transitions, insufficient meeting duration, repeated visits, invalid plan formatting, or a non-optimal objective value when optimality is part of SR.
- **Exact-match feedback:** valid plans whose normalized text does not match the development reference plan. This signal is used only after a candidate plan has been generated; it is not an input to the solver that constructs the plan.

Routing rule. The feedback packet is routed according to the likely source of the failure (see the Figure 6). If the failure is local to code mechanics, such as a parser mistake, formatting drift, runtime exception, or verifier mismatch, the builder patches the implementation while preserving the current architecture. If the failure reveals that the current abstraction cannot express the domain behavior, such as a wrong preference interpretation, an inadequate search objective, or an incorrect tie-breaking rule, the architect revises Φ and sends a revised specification to the builder. The runner then reruns the same development gate on the patched program. Thus, the runner does not directly change the solver; it produces the evidence that determines whether the next action is an implementation repair or an architectural revision.

Calendar scheduling evidence. The calendar-scheduling implementation follows this protocol in `calendar_scheduling/`. The architect file defines an interval-based representation with day order, work-window bounds, busy intervals, preference exclusions, and candidate meeting intervals. The builder implements the corresponding parser, earliest-feasible-slot solver, renderer, and verifier. The runner treats the builder as a black box and returns a status and plan for each selected instance. The evaluator exposes the development gate through the `-dev` option, which restricts execution to the first 20 instances. Success rate is computed by checking the generated plan against parsed prompt constraints; exact match is computed only afterward by normalizing the generated plan and the stored `golden_plan`. The domain-specific interaction diagram visualizes this process, where the runner records parser, runtime, constraint, and EM mismatches, sends the failure evidence back

A.3 Development-Time Feedback in PLANFORGE

This appendix expands the iterative feedback loop described in Section 3.4. The central point is that feedback is a development-time control signal; during synthesis, the runner repeatedly executes

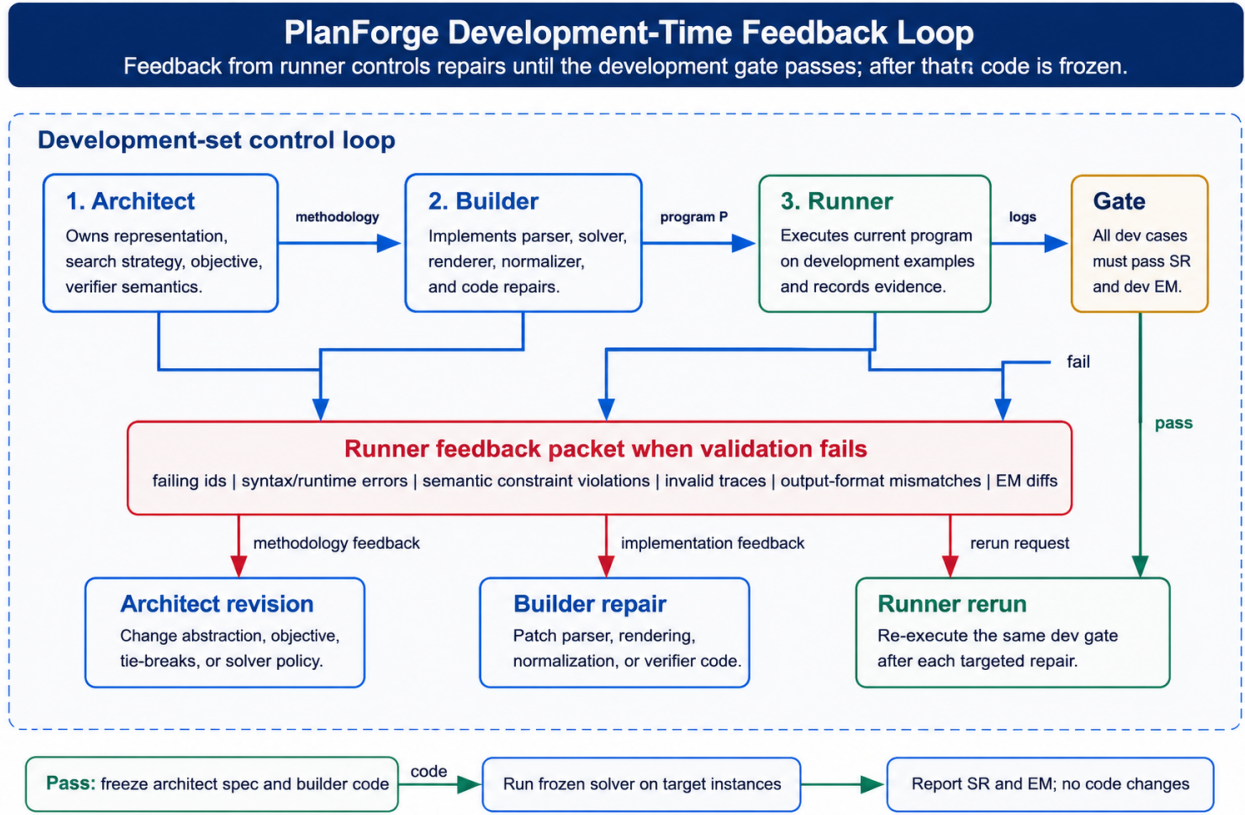


Figure 6: Generalized development-time feedback loop used by PLANFORGE. The architect proposes the planning methodology, the builder implements it as executable code, and the runner executes the code on the development instances. Failures are returned as typed feedback packets and routed either to the builder for code repair or to the architect for a methodology revision. Once the development gate passes, the code is frozen and the same program is used for evaluation.

through the feedback router, and the system freezes the architect and builder artifacts only after the 20-instance development gate passes.

Meeting planning evidence. The meeting-planning implementation in `meeting_planning/` uses the same feedback contract with a different domain abstraction. The architect defines a structured state consisting of the start location and time, friend availability windows, travel-time transitions, and a visited-friend mask. The objective is to maximize the number of meetings with deterministic tie-breaking. The builder implements dynamic programming over this representation and renders the resulting route as a plan. The evaluator verifies SR without using the reference plan, as it reconstructs the meeting steps, checks travel and time-window consistency, rejects repeated visits, and confirms that the number of meetings matches the dynamic-programming optimum. EM is then computed by comparing the normalized generated plan with `golden_plan`. The figure shows the same development-set control loop, where failures return as packets containing instance ids, constraint violations, exceptions, invalid timing traces, output-format mismatches, or EM diffs; a 20/20 development pass triggers code freeze.

Generalization across domains. The same mechanism applies beyond these two NaturalPlan domains. A new domain only changes the contents of Φ , the builder functions, and the semantic verifier. The control structure remains fixed, using development instances to generate typed runner feedback; route implementation failures to

the builder; route representational, objective, or search failures to the architect; rerun the same development gate after each repair; and freeze the resulting program before full evaluation. This separation is what prevents the feedback loop from becoming ad hoc test-set tuning. During development, feedback is used to repair a domain-general executable planner for the observed development distribution. During evaluation, the planner is held fixed and is measured by SR and EM on the target instances.

B Architect-Selected Solution Strategies

This section will show how the architect selects an appropriate computational formulation for each benchmark family: search for Plan Generation, constraint solving for NaturalPlan, and satisfiability solving for StructureSAT. It will include one compact representative blueprint rather than exhaustive domain-specific artifacts.

NaturalPlan This section summarizes the blueprints produced by the Architect for the three NaturalPlan domains. In our Architect–Builder–Runner setup, the Architect is not asked only to describe the task. It must specify an executable planning design: the representation of the problem, the functions that the Builder should implement, the algorithmic strategy for finding a plan, and the logic that connects parsing, solving, validation, and output generation. The following sections make these domain-specific blueprints explicit.

B.1 Trip Planning

The Architect treats trip planning as a constrained path-finding problem over a flight graph. Cities are graph nodes, direct flights are directed edges, and a complete itinerary is an ordered path that visits every required city exactly once. The temporal structure is represented by inclusive visit intervals. If the traveler flies from one city to the next on day d , then day d is shared by the two consecutive city intervals. Therefore, an itinerary with n cities must satisfy the total-day identity $\sum_i \text{duration}_i - (n - 1)$.

Trip Planning blueprint:

- **State representation:** a problem instance contains the city list, required stay duration for each city, total trip length, directed flight edges, and fixed event windows. A partial or complete plan is a sequence of visit records (*city, start, end*). The state must preserve city coverage, valid flight connectivity, inclusive day intervals, and the requirement that every fixed event window lies inside the visit interval for its city.
- **Core functions:** the Builder is expected to implement parsing routines for cities, durations, total days, flight edges, and event constraints; candidate-generation logic for extending a partial route to feasible next cities; consistency checks for total-duration arithmetic; a route verifier that checks city coverage, flights, intervals, and events; and a formatter that converts the verified route into a day-by-day trip plan.
- **Solving strategy:** the Architect specifies deterministic constrained depth-first search over possible flight paths. The search prunes a branch when the current city interval violates an event window, exceeds the trip horizon, or makes a remaining city impossible to schedule later. For small unconstrained cases, the blueprint permits a deterministic permutation-based fallback over city orders.
- **Integration logic:** the system first parses the natural language prompt into a structured graph-and-interval problem. It then checks the total-day arithmetic induced by shared flight days, searches for a feasible city ordering, verifies the resulting itinerary against all parsed constraints, and returns the plan only after this verification succeeds.

Meeting Planning The Architect treats meeting planning as a time-window routing problem. The agent starts at a specified

location and time, then chooses an ordered sequence of friends to meet. Each transition must account for travel time, possible waiting, the friend's availability window, and the minimum required meeting duration. Unlike trip planning, the goal is not necessarily to visit every friend; the objective is to maximize the number of feasible meetings.

meeting planning blueprint:

- **State representation:** a planning state consists of the current location, current clock time, and set of already-met friends. Each friend is represented by a name, location, availability interval, meeting duration, and stable input index. The travel-time matrix defines the cost of moving between locations. A candidate meeting is valid only if the agent can arrive, wait if needed, and complete the meeting before the friend's availability window closes.
- **Core functions:** the Builder is expected to implement time parsing and formatting, availability-window parsing, construction of the structured meeting problem, travel-time lookup, candidate generation over unmet friends, recursive schedule search, tie-breaking logic, schedule verification, and natural-language plan formatting. Each candidate action records the departure location, travel time, arrival time, meeting start, and meeting end.
- **Solving strategy:** the Architect specifies exact dynamic search with memoization over subset-style routing states. From each state, the solver evaluates all friends who have not yet been met and recursively computes the best continuation. The primary objective is to maximize the number of meetings. Ties are resolved deterministically, favoring earlier completion, less waiting, and stable input order.
- **Integration logic:** the system parses the start state, friend constraints, and travel matrix; computes the best feasible meeting sequence; replays the sequence to verify travel times, arrival times, waiting, availability windows, meeting durations, and duplicate-meeting violations; and finally emits the verified route as an ordered list of plan steps.

Calendar Scheduling The Architect treats calendar scheduling as finite interval enumeration over an ordered work-calendar grid. The problem is to choose one meeting slot that fits within the allowed days and work hours while avoiding all participant conflicts and respecting any stated preferences. Because the candidate days, work

hours, and meeting duration are bounded, the Architect chooses direct constraint enumeration rather than a more complex search procedure.

Calendar scheduling blueprint:

- **State representation:** a calendar instance contains the participants, ordered candidate days, work-hour start and end times, required meeting duration, per-participant busy intervals, blocked days, lower time bounds, upper time bounds, and an earliest-availability preference. A candidate plan is a tuple *(day, start, end)*, where the interval length must equal the requested duration.
- **Core functions:** the Builder is expected to implement clock parsing, duration parsing, day extraction, participant extraction, busy-interval parsing, interval merging, preference parsing, candidate-slot generation, slot-validity checking, result verification, and output formatting. The candidate generator scans possible starts on a deterministic 30-minute grid.
- **Solving strategy:** the Architect specifies chronological constraint enumeration. For each allowed day, the solver enumerates candidate start times and rejects a slot if it falls outside work hours, overlaps any participant's busy interval, occurs on a blocked day, starts before a parsed lower bound, or ends after a parsed upper bound. In the default case, the first feasible slot is returned; when the parsed preferences call for a later free block, the solver deterministically selects the beginning of the last available block.
- **Integration logic:** the system parses the prompt into a structured calendar problem, normalizes busy intervals by merging overlaps, enumerates valid slots in day and time order, constructs the proposed meeting time, verifies its duration and participant-level constraints, and returns the final slot only if all checks pass.

Overall, the Architect's blueprints identify three distinct algorithmic structures for NaturalPlan: constrained graph search for trip planning, memoized time-window routing for meeting planning, and interval-based slot enumeration for calendar scheduling. These blueprints make clear what the Architect actually specifies: the planning state, the required parsing and validation functions, the candidate-generation mechanism, the solving algorithm, and the final integration path from natural-language prompt to verified plan.

B.2 Plan Generation (Plan Gen.)

The following summarize what the Architect specified for each 15 doomains in Plan Gen. which is part of ACPBench domain. Each box is grounded in the implemented domain code and highlights the state representation, functions to implement, solving strategy, and integration logic used by the Plan Gen. pipeline.

ALFWorld

Architect output. For ALFWorld, the Architect represents the household as an AWState: agent location, typed objects and receptacles, object locations, object-receptacle containment, open/closed receptacles, toggled objects, validated task facts, and an optional held object. Goals are separated into direct symbolic facts and higher-level validation goals such as placing, cleaning, heating, cooling, or examining an object. The Builder-facing functions are `parse_instance`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is a deterministic symbolic task-template planner. It selects available objects of the required type, chooses feasible receptacles or processors, opens containers when needed, navigates the agent to the right location, performs pickup/put/toggle/process actions, and appends explicit validation actions. The integration logic is `parser-to-state-to-planner-to-simulator`: natural language facts are normalized into the Architect state, the planner emits ALFWorld action strings, the runner simulates those actions, and `goals_hold` checks that all direct and validation-style goals are satisfied.

BlocksWorld

Architect output. For BlocksWorld, the Architect uses BWState, containing the block list, an on child-to-support map, an ontable set, and the optional block held by the arm. Goals are stored in BWGoals: desired on pairs, clear blocks, table blocks, holding/handempty facts, and conflict flags. The required functions are `parse_instance`, `validate_goal_consistency`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is constructive: reject impossible goal cycles or mutually exclusive holding/table/clear facts, put down any initially held block, clear existing stacks to the table, then rebuild the requested goal stacks from their bases

BlocksWorld (continued)

upward.

The integration logic follows the standard BlocksWorld operators pick-up, put-down, stack, and unstack. The runner parses natural language, invokes the constructive planner, replays the action list with precondition checks, and validates the resulting symbolic state against the goals.

Depot

Architect output. For Depot, the Architect represents typed PDDL objects in DepotState: crates, pallets, hoists, trucks, places, object locations, available hoists, clear surfaces, crate stacking, crates in trucks, and hoist lifting status. Goals are kept as structured fact tuples in DepotGoals. The implementation functions include `parse_instance`, `validate_goal_consistency`, `construct_plan`, `simulate_plan`, and `goals_hold`, with helper routines for lifting, dropping, loading, unloading, driving trucks, freeing hoists, and building crate stacks. The solving strategy first validates contradictions such as stack cycles, then constructs requested on relations and satisfies remaining at, in, clear, available, and lifting facts. The integration logic relies on exact PDDL parsing for the initial and goal facts. The planner emits depot actions, the simulator applies the same operator semantics, and the runner reports a plan only when the replayed final state satisfies the Architect's goal facts.

Ferry

Architect output. For Ferry, the Architect selects FerryState: cars, locations, a car-to-location map for cars ashore, the ferry location, and an optional onboard car. Goals are represented by FerryGoals, including car destinations, ferry destination, onboard/empty requirements, and explicit conflict flags. The Builder implements `parse_instance`, `validate_goal_consistency`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is deterministic transport over fully connected locations: empty the ferry if needed, sail to a car, board it, sail to the target location, debark it, and then

Ferry (continued)

satisfy any final ferry/onboard/empty condition. The integration logic exposes only legal ferry actions: sail, board, and debark. The runner parses the natural language instance, generates action strings through the planner, simulates ferry capacity and location constraints, and accepts the result only when the final state satisfies all goals.

FloorTile

Architect output. For FloorTile, the Architect uses FloorTileState: colors, robots, tiles, directed movement edges, robot locations, robot paint colors, clear tiles, and already painted tile colors. Goals are represented as structured PDDL fact tuples in FloorTileGoals. The key functions are `parse_instance`, `validate_goal_consistency`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy combines goal mutex validation with informed forward state-space search. The code includes a painting-specific constructive attempt, path planning for robots, moving blocking robots aside, and an A* fallback over movement and paint successors. The integration logic maps PDDL facts into a compact grid/robot state, plans legal move and paint actions, replays them under tile occupancy and adjacency constraints, and checks that the required painted facts hold in the final state.

Frogs Jumping

Architect output. For Frogs Jumping, the Architect represents the puzzle as an immutable one-dimensional FrogState: board tuple, frog-position pairs, left-facing frog set, and right-facing frog set. Goals in FrogGoals contain requested frog positions and empty-pad facts. The required functions are `parse_instance`, `construct_plan`, `simulate_plan`, and `goals_hold`, supported by legal move generation, static impossibility checks, canonical continuation, and best-first search. The strategy respects monotone directional movement: frogs slide or jump only in their facing direction, and jumps are allowed only over the correct adjacent occupied pad.

Frogs Jumping (continued)

The integration logic parses the natural language board into indexed pads, emits slide/jump action strings, simulates every action against the board, and validates that the final board satisfies the requested partial or complete configuration.

GoldMiner

Architect output. For GoldMiner, the Architect uses GMState: grid dimensions, robot location, held item, bomb and laser locations, clear cells, soft rocks, hard rocks, and gold cells. Goals are normalized as predicate/location atoms in GMGoals.

The Builder implements `parse_instance`, `construct_plan`, `simulate_plan`, and `goals_hold`, with helper operators for movement, laser pickup/drop, bomb pickup/detonation, firing the laser, and picking gold. The solving strategy is deterministic best-first graph search with additional constructive routines for reaching gold, carving paths through obstacles, and satisfying post-gold conditions.

The integration logic turns natural language grid facts into cell sets, searches over executable domain transitions, emits action strings, replays the destructive/resource actions in the simulator, and verifies the normalized goal atoms in the final state.

Grid

Architect output. For Grid, the Architect uses GridState: rows, columns, robot location, optional held key, key locations, key shapes, lock shapes, and locked cells. Goals are normalized into GridGoals, covering robot position, holding, key-at, arm-empty, open, and locked facts.

The functions to implement are `parse_instance`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is A* graph search over deterministic STRIPS-style transitions with monotonic unlock effects, plus constructive key-delivery routines that choose keys, navigate to keys, unlock matching locks, and place keys at target cells.

The integration logic parses grid dimensions, keys, locks, and goals from text; the planner emits move, pickup, putdown, and unlock actions; the

Grid (continued)

simulator enforces adjacency, holding, and key-shape constraints; and the runner validates the final symbolic grid state.

Grippers

Architect output. For Grippers, the Architect specifies GrippersState: robots, rooms, balls, grippers, robot locations, ball locations, carried balls per gripper, and free grippers. Goals are represented in GrippersGoals, with requested robot locations, ball locations, carrying/free facts, and contradiction flags.

The Builder implements `parse_instance`, `validate_goal_consistency`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is constructive in a fully connected room graph: detect impossible requests, move the robot to each ball, pick it with a free gripper, move to the target room, drop it, and then satisfy any remaining carry/free/robot-position goals.

The integration logic uses only standard grippers actions move, pick, and drop. The runner parses the instance, calls the constructive planner, simulates gripper capacity and ball uniqueness constraints, and checks the final state against the Architect goals.

Hanoi

Architect output. For Hanoi, the Architect represents the puzzle as HanoiState: ordered disks, pegs, and an on map from each disk to its current support. Goals in HanoiGoals contain requested direct on facts and clear objects.

The functions are `parse_instance`, `validate_goal_consistency`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy first rejects illegal goal cycles and size violations, then uses deterministic recursive tower movement when the goal is a tower transfer and bounded search for smaller partial configurations. The integration logic parses disk/peg relations from the text, emits move actions, simulates top-disk and size preconditions, and verifies final on and clear facts. This makes the classical Hanoi constraints explicit rather than relying on a generic textual answer.

Logistics

Architect output. For Logistics, the Architect uses `LogisticsState`: cities, locations, packages, trucks, airplanes, location-to-city mapping, each city's airport, object locations, and package-in-vehicle relations. Goals are stored in `LogisticsGoals`, including at and in facts plus contradiction flags.

The required functions are `parse_instance`, `validate_goal_consistency`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is a deterministic constructive logistics planner: trucks move within cities, airplanes move between airports, and packages are routed through airports for inter-city delivery while direct intra-city moves use trucks. The integration logic parses city topology and current object placements, emits drive-truck, fly-airplane, load-truck, unload-truck, load-airplane, and unload-airplane actions, simulates vehicle containment and city restrictions, and accepts only plans whose final state satisfies all package/vehicle goals.

Rovers

Architect output. For Rovers, the Architect uses a structured PDDL fact model: `RoverProblem` stores typed objects, initial facts, and goal facts; `RoverState` expands these into rover locations, lander locations, stores, camera/objective/mode data, samples, communicated data, images, calibration facts, and static traversability/visibility/support facts. The core functions are `parse_instance`, `make_state`, `construct_plan`, `simulate_plan`, and `goals_hold`, with helpers for navigation, sample analysis, image acquisition, communication, and action application. The solving strategy is goal-directed constructive planning over STRIPS facts: BFS finds traversable rover paths, then the planner builds prerequisites for soil, rock, image, and communication goals before executing the matching action.

The integration logic parses exact PDDL sections, protects already achieved goals where needed, emits rover-domain actions, simulates each operator with precondition checks, and validates all goal facts against the reconstructed final rover state.

Satellite

Architect output. For Satellite, the Architect uses `SatelliteState`: directions, instruments, modes, satellites, instrument-to-satellite mapping, supported modes, calibration targets, current pointing, available power, powered instruments, calibrated instruments, and acquired images. Goals are represented as structured fact tuples.

The Builder-facing methods are `parse_instance`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is constructive resource planning: reject mutually exclusive pointing goals, choose an instrument that supports the requested image mode, make power available, switch instruments on/off, turn the satellite to calibration targets, calibrate, turn to image targets, and take images. The integration logic emits the Architect's action schemas: `turn_to`, `switch_on`, `switch_off`, `calibrate`, and `take_image`. The runner parses PDDL or input-derived goals, replays the plan under power/calibration/pointing constraints, and validates that all image, pointing, power, and calibration goals hold.

Swap

Architect output. For Swap, the Architect represents the world as a finite bijection in `SwapState`: sorted agent-to-role assignments, the set of agents, and the set of roles. Goals in `SwapGoals` are requested assignment atoms.

The required functions are `parse_instance`, `construct_plan`, `simulate_plan`, and `goals_hold`. The solving strategy is deterministic cycle repair by legal pairwise swaps. The planner rejects unknown agents or roles, multiple target roles for the same agent, multiple agents targeting the same role, and non-bijective initial assignments; then it repeatedly finds the current holder of a target role and swaps the two assignments.

The integration logic parses assignments from natural language, emits `swap agent1 agent2 role1 role2` actions, simulates each pairwise exchange with precondition checks, and verifies that every requested assignment atom holds in the final bijection.

VisitAll

Architect output. For VisitAll, the Architect uses VisitAllState: grid width and height, available locations, unavailable cells, adjacency map, current robot location, and monotonic visited set. Goals in VisitAllGoals include required visited locations, an optional final robot location, and an at-conflict flag. The Builder implements parse_instance, validate_goal_consistency, construct_plan, simulate_plan, and goals_hold. The solving strategy is deterministic graph route construction: validate that all goal cells are available and reachable, then repeatedly use BFS to move to the nearest unvisited required location and finally route to the requested final location if one is specified. The integration logic constructs grid adjacency from the parsed dimensions and blocked cells, emits move source destination actions, simulates each move over the adjacency map while updating the visited set, and validates that every required location has been visited.

B.3 StructureSAT

This appendix summarizes the Architect-specified blueprints produced by PLANFORGE for the nine StructureSAT domains. In the PLANFORGE Architect-Builder-Runner workflow, the Architect does not emit a prediction rule or a hand-written answer key. Instead, it initializes and then freezes a methodological blueprint: the instance representation, the solving strategy, the branching rule, and the validation condition that the Builder must implement. The Runner then accepts the blueprint only after the development instances are solved successfully, after which the same frozen implementation is evaluated without further Architect revisions. Thus, the blueprints are intended to document the algorithmic basis of the system, not to introduce a new solver family.

3-SAT

For the 3-SAT domain, the Architect specifies a direct DIMACS-CNF state representation. Each clause is stored as a tuple of integer literals, where a positive literal requires the corresponding Boolean variable to be true and a negative literal requires it to be false. The active search state is a variable-to-Boolean assignment together with the unresolved clauses. The Builder functions implied by this blueprint are: parse the DIMACS file, simplify clauses under a partial assignment, propagate unit clauses, eliminate pure literals, choose a branch variable, recursively run DPLL,

3-SAT (continued)

and verify any returned satisfying assignment. The solving strategy is complete recursive DPLL with fixed-point propagation, contradiction detection, and chronological backtracking. Branching is ordered by choosing an unassigned variable that is frequent in the shortest unresolved clauses and trying the majority polarity first. The Runner integration parses each instance, calls the Builder solver, accepts SAT only when every clause is satisfied by the returned assignment, and accepts UNSAT only when the complete search exhausts all branches.

Automorphism

For the automorphism domain, the Architect recognizes that many instances are not merely arbitrary CNFs but square exact-one permutation-matrix encodings. The primary state representation is therefore a permutation CSP: each row must choose exactly one column, each column must be used exactly once, and binary negative clauses become incompatibility constraints between candidate row-column choices. The same DIMACS variable ids are retained so that every CSP solution maps back to a Boolean SAT assignment. The Builder functions implied by this blueprint are: parse DIMACS, detect the square permutation encoding, build row and column exact-one constraints, build the incompatibility graph from binary negative clauses, compute feasible columns, perform forward checking, memoize failed partial permutations, and fall back to the general DPLL solver when the structure is not present. The solving strategy is complete permutation backtracking with minimum-feasible-row branching and support-based column ordering; for nonmatching CNFs it uses the same complete DPLL procedure as the other domains. The Runner integration validates the final Boolean assignment against the original clauses, so the structured CSP path is accepted only when it satisfies the DIMACS formula.

Cellular Automata

For the cellular automata domain, the Architect keeps the representation at the CNF level supplied by StructureSAT. Each instance is parsed into immutable tuples of DIMACS literals, and the solver state is the current partial

Cellular Automata (continued)

Boolean assignment plus the clauses not yet satisfied or contradicted. The Builder functions implied by this blueprint are: parse and validate the DIMACS header and clauses, simplify clauses after each assignment, repeatedly apply unit propagation and pure literal elimination, detect empty-clause conflicts, select branch variables, and certify SAT assignments. The solving strategy is complete DPLL search with chronological backtracking. The branching rule focuses on the shortest unresolved clauses and picks the highest-occurrence unassigned variable, trying the locally dominant polarity first. The Runner integration deliberately does not use filenames, label folders, or cellular-automata metadata for prediction; labels are used only after inference to compute accuracy.

k -Clique

For the k -clique domain, the Architect treats the graph problem through its already-provided SAT encoding. The state representation is the DIMACS CNF formula, not a reconstructed graph: literals are integer-coded Boolean variables, clauses are immutable tuples, and search states are partial variable assignments.

The Builder functions implied by this blueprint are: parse the CNF instance, evaluate whether clauses are satisfied, unresolved, or contradicted, propagate forced literals, eliminate pure literals, choose a branching variable, and run complete recursive DPLL. The solving strategy is a standard exact CNF SAT search; it does not guess clique existence from dataset names or labels. Branching is only a search-ordering heuristic, prioritizing variables that occur most often in the currently shortest unresolved clauses. The Runner integration accepts a SAT result only with a clause-satisfying assignment and accepts UNSAT only after exhaustive backtracking proves that no assignment exists.

k -Dominating Set

For the k -dominating-set domain, the Architect again specifies a formula-only state representation. Each instance is represented as DIMACS CNF clauses and

k -Dominating Set (continued)

a variable-to-Boolean search state. The blueprint intentionally ignores graph generator metadata, file naming conventions, and SAT/UNSAT directory labels during solving. The Builder functions implied by this blueprint are: parse DIMACS into a formula object, reduce the formula under partial assignments, compute unit and pure-literal implications, detect conflicts, select branches from short unresolved clauses, and check returned assignments. The solving strategy is complete DPLL with repeated simplification, propagation, pure literal elimination, and chronological backtracking. The Architect selected exact search because the development instances were small enough for complete reasoning and because exact search avoids shortcut rules. The Runner integration freezes the method after development success, evaluates the same implementation on held-out instances, and reports accuracy against labels only after predictions are produced.

k -Vertex Cover

For the k -vertex-cover domain, the Architect observes that the files are ordinary DIMACS CNF formulas, including mixed filename suffixes. The state representation therefore normalizes all files to the same formula object: tuples of integer literals plus a partial assignment over the declared variables. The Builder functions implied by this blueprint are: parse valid DIMACS CNF despite filename variations, simplify formulas, run fixed-point propagation, apply pure literal elimination, detect contradictions, branch on informative unassigned variables, and validate solutions. The solving strategy is complete DPLL search. Variables are chosen from shortest unresolved clauses by occurrence count, and the majority polarity is tried first before backtracking. The Runner integration treats filename suffixes and label directories as non-semantic for solving; they are not part of the prediction rule. A prediction is accepted by formula-level validation, not by matching a path pattern.

***k*-Coloring**

For the k -coloring domain, the Architect specifies the same exact SAT-level representation used for the graph-derived domains. The coloring constraints have already been compiled into DIMACS clauses, so the Builder works with integer literals and Boolean assignments rather than rebuilding a separate coloring data structure. The Builder functions implied by this blueprint are: parse the CNF, maintain a partial assignment, simplify unresolved clauses, propagate unit clauses, remove pure literals, identify conflicts, branch recursively, and confirm that SAT assignments satisfy all clauses. The solving strategy is complete DPLL with chronological backtracking. The branching heuristic selects a variable with high occurrence among shortest unresolved clauses and tries the more common polarity first. The Runner integration uses the frozen Builder solver for both development and evaluation, with dataset labels used only for post-hoc SAT/UNSAT accuracy.

SR

For the SR domain, the Architect describes the development instances as small-to-moderate DIMACS CNF formulas and therefore chooses a complete CNF SAT blueprint rather than a random classifier, label shortcut, or domain-name rule. The state representation is a tuple-based CNF formula plus a variable-to-Boolean assignment. The Builder functions implied by this blueprint are: parse the DIMACS formula, evaluate clause satisfaction under partial assignments, simplify unresolved clauses, propagate forced literals, eliminate pure literals, choose branch variables from short clauses, recurse with backtracking, and verify any SAT witness. The solving strategy is complete DPLL with the same occurrence-based branch ordering used in the other CNF domains. The Runner integration freezes the Architect specification after the development split succeeds, uses the same Builder implementation for evaluation, and accepts results only through formula-level SAT validation or exhaustive UNSAT search.

PS

For the PS domain, the Architect represents each instance directly as a DIMACS CNF formula. Positive and negative literals retain their standard Boolean meanings, and the solver state is a partial mapping from variable ids to truth values. The Builder functions implied by this blueprint are: parse the input CNF, validate literal ranges and clause counts, simplify clauses, perform unit propagation, apply pure literal elimination, choose branch variables, run recursive DPLL, and stringify or return assignments for Runner output. The solving strategy is a complete SAT procedure with contradiction detection and chronological backtracking; it does not infer predictions from ordering, filenames, or label folders. The Runner integration catches parsing or solving errors as failures, records runtime and decision counts, checks returned SAT assignments against every clause, and computes correctness only after comparing the prediction with the dataset label.

B.4 Representative Blueprint

We are providing the verbatim blueprint only for one domain: the `trip_planning` domain from NaturalPlan. The text below is extracted directly from the ABR interaction log file for this domain, specifically from the architect's blueprint section. It records the architect's initial design for the state representation, functions and data structures to implement, solving strategy, and integration logic used by the builder and runner.

trip Planning domain Blueprint

Represent each instance as: 1. cities: ordered tuple from the dataset cities field; 2. durations: map from city to required inclusive day count; 3. total_days: integer parsed from prompt_0shot; 4. flights: directed edge set parsed from the direct-flight section; 5. events: city-specific inclusive day windows parsed from prompt_0shot.
Use this planning rule: For a route $c_1, c_2, \dots, c_n$, c_1 starts on day 1. A city c_i with duration d occupies days start through start + $d - 1$. The next city starts on the same day that the previous city ends because flying consumes the boundary day. The route is valid only when the final end day equals total_days.

trip Planning domain Blueprint (continued)

Use this solving rule: Search over complete city routes using DP state (visited_mask, current_city). The next start day is determined by the set of visited cities: $\text{sum}(\text{durations of visited cities}) - (\text{number of visited cities} - 1) + 1$. Try next cities in dataset order. Reject a transition if no direct flight exists from current_city to next_city. Reject a city placement if any event window for that city is not fully contained in the candidate interval.

Use this rendering rule: Return the NaturalPlan answer text: Here is the trip plan for visiting the N European cities for T days:

```

**Day s-e:** Arriving in CITY and visit
CITY for D days. **Day e:** Fly from
CITY to NEXT. **Day s-e:** Visit CITY
for D days.

```

C Additional Experimental Evidence

This section provides supplementary empirical evidence supporting the main claims of the paper: execution-based failure modes, performance scaling as plan horizon and object complexity increase, and computational token cost and code-reuse efficiency.

C.1 Execution Failure Taxonomy and Diagnostic Analysis

To rigorously diagnose why direct LLM plan generation fails and how PLANFORGE mitigates reasoning errors, we perform an execution-based error analysis across all 15 evaluated domain benchmarks. Standard textual metrics (such as string overlap or BLEU score) and answer-only reachability verdicts (yes/no) fail to expose operational flaws in generated plans. For instance, a model may correctly predict that a plan exists, but produce an action sequence that assumes invalid initial conditions, violates state preconditions mid-execution, or fails to fulfill the complete goal.

We categorize all execution failures into six fine-grained error classes, as defined in Table 8. Specifically, **Initial-State Mismatch** and **Goal-State Mismatch** occur when the generated plan or state translator assumes an incorrect environment starting state or target goal condition. **Precondition Violation** occurs when an action is executed before its required state conditions hold, whereas **Invalid Action** refers to hallucinated or ill-formed action names. Finally, **Complete Goal Mismatch** describes trajectories that run without runtime errors but end in a non-goal state, while **Partial Goal Match** occurs when only a subset of the goal propositions is satisfied.

Comparative Error Distributions: Direct Generation vs. PLANFORGE Figure ?? illustrates the aggregate distribution of execution failures across all evaluated benchmarks for direct baseline models versus PLANFORGE. As shown in Figure 7 and Table 9, direct generation (CoT) suffers heavily from precondition violations (over 38% of errors) and invalid action hallucinations because sequential token generation lacks transition dynamics verification. In contrast, Figure 8 demonstrates that PLANFORGE virtually eliminates mid-execution precondition violations and invalid actions. Because PLANFORGE executes a synthesized deterministic search algorithm, every emitted plan strictly satisfies domain transition rules, leaving

Failure type	Interpretation
Initial-state mismatch	The plan or verdict assumes an initial state different from the true instance.
Goal-state mismatch	The plan or verdict is evaluated against an incorrect or incomplete goal condition.
Precondition violation	An action is applied when its required preconditions are false in the current state.
Invalid action	The sequence contains a hallucinated or unsupported action not present in the domain.
Complete goal mismatch	Execution terminates in a state that does not satisfy the goal condition.
Partial goal match	Execution reaches a state satisfying only part of the goal, but not the full goal.

Table 8: **Execution-based failure taxonomy.** Execution reveals whether a generated plan is grounded in the actual transition dynamics or merely syntactically plausible.

residual errors concentrated almost entirely in early state translation (NL2STATE).

Diagnostic Implications This taxonomy underscores why PLANFORGE achieves superior reliability. Rather than relying on fragile next-token prediction to maintain state invariants across long horizons, PLANFORGE shifts LLMs to program synthesis where state transitions are enforced by execution. When PLANFORGE does fail, the error is isolated to the state translation module (NL2STATE) or the domain solver design, making failures modular, transparent, and amenable to targeted debugging.

C.2 Plan Gen: Long-Horizon Scaling Analysis

A central question in neural planning is whether language model performance degrades as problem complexity and required plan horizon scale. In standard step-by-step autoregressive generation (e.g., Chain-of-Thought), accuracy decays rapidly over long horizons due to compounding error rates, context window drift, and attention dispersion over complex state histories. To evaluate long-horizon robustness, we conduct an empirical scaling analysis on **Plan Gen**, our extended ACPBench-based dataset spanning 15 planning domains with ground-truth plan lengths ranging from single-step operations to over 600 actions (e.g., *frogs_jumping* up to 624 steps, *hanoi* up to 511 steps, and *blocksworld* up to 368 steps).

This section provides a detailed analysis of how planning performance scales with increasing plan length and problem complexity. We evaluate **Plan Gen**, our extended ACPBench-based setting, to study whether planner performance remains reliable as plan length and problem complexity increase.

Goal. We aim to answer a key question:

Does increasing plan length degrade planning accuracy?

We analyze three aspects: (i) dataset statistics, (ii) plan existence prediction behavior, and (iii) VAL-verified plan generation accuracy.

Dataset Characteristics Figure 9 provides understanding about Plan Length and No of objects involved across the instances of Plan Gen. dataset across all the 15 domains. where as Table 10 summarizes domain-level plan statistics, with accuracy using VAL validator across all the Existed Plans for the instances in Plan Gen. benchmark.

Observation. The benchmark includes extremely long plans (up to 624 steps), making it suitable for testing scalability.

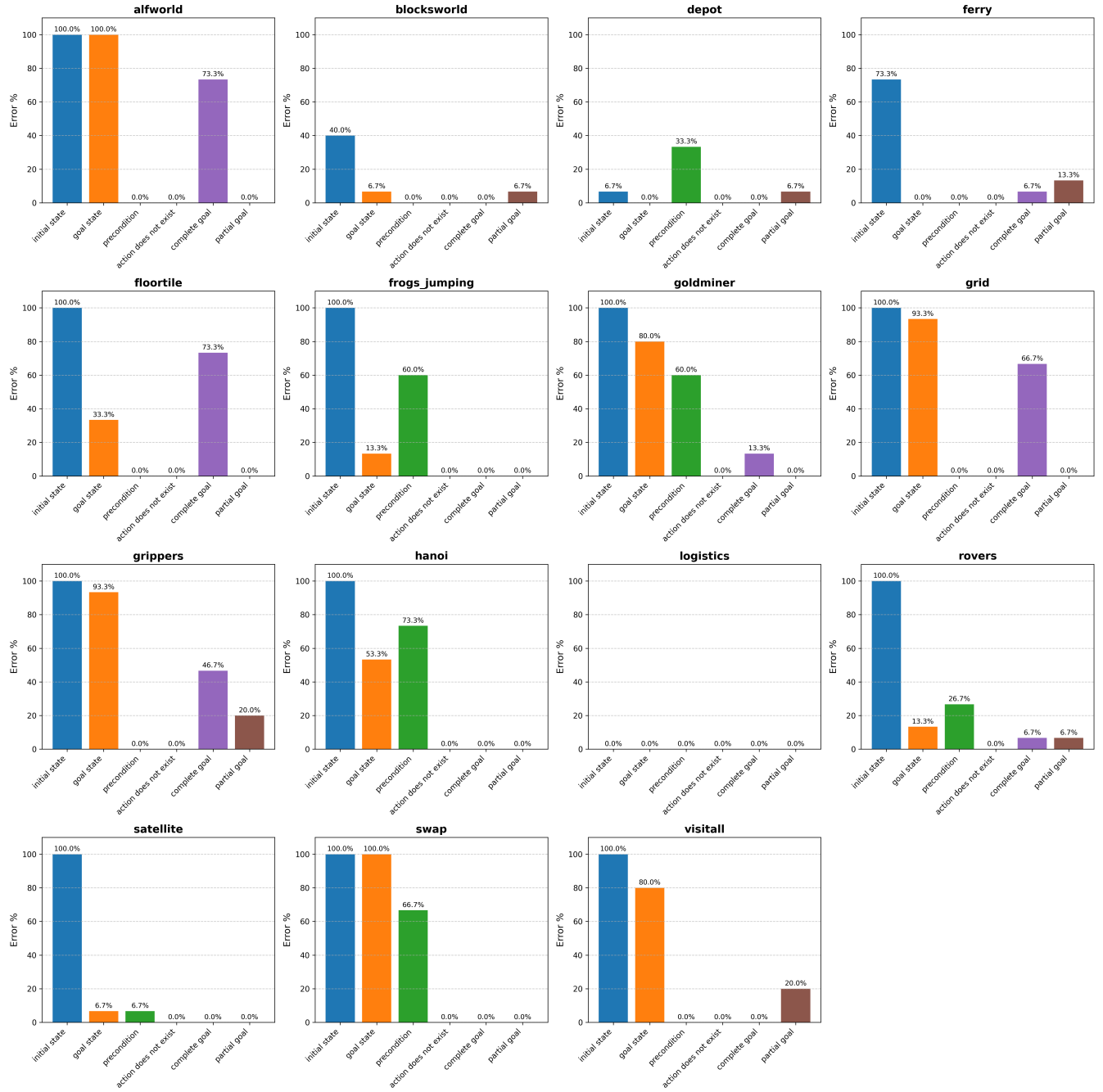


Figure 7: Domain-wise Error Analysis (CoT). Error distribution for the Baseline model across 15 PDDL domains. Each subplot represents the percentage of failed instances categorized by the type of violation or mismatch, illustrating domain-specific performance variability in standard VLM-based planning.

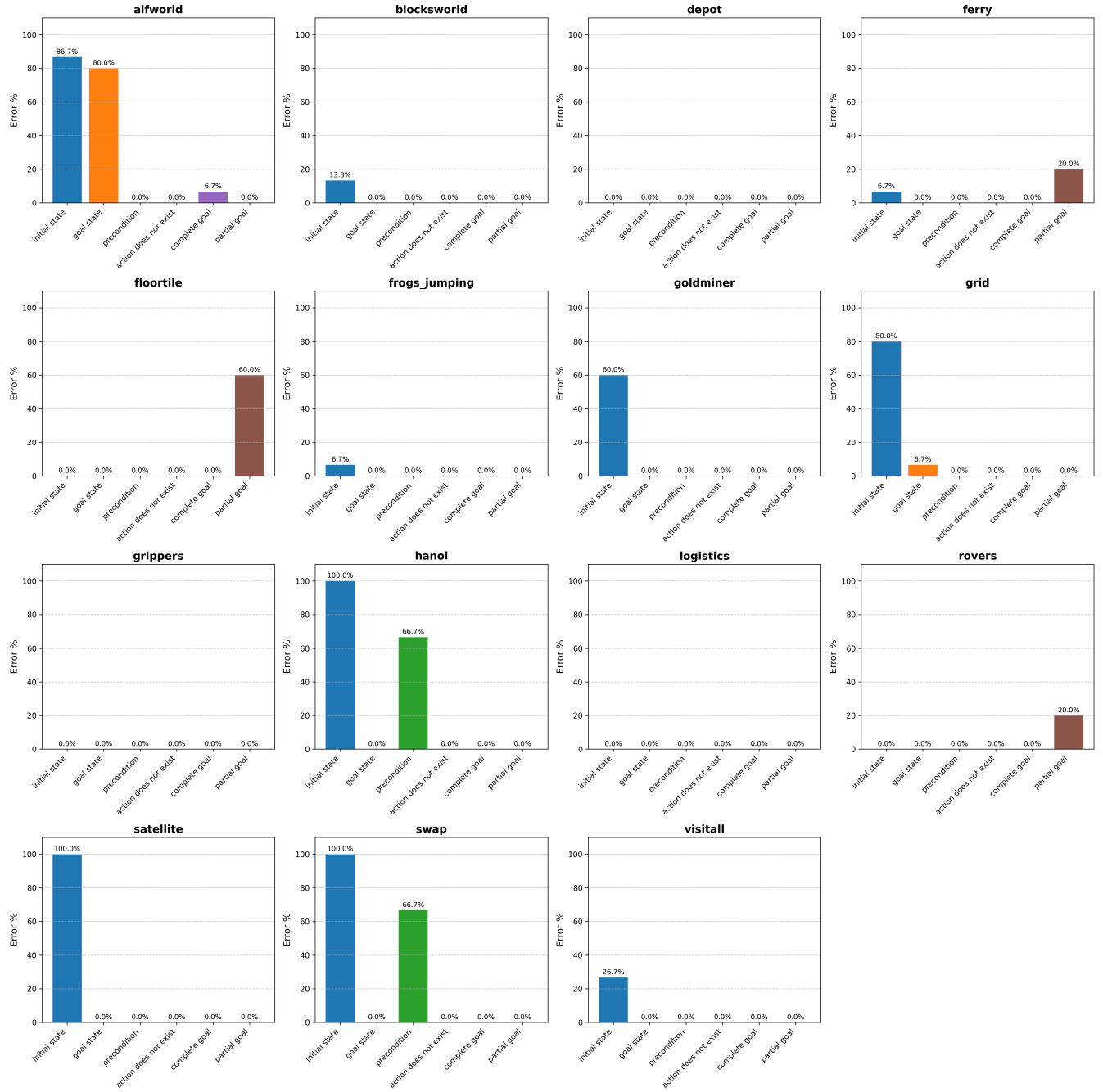


Figure 8: Domain-wise Error Analysis (PLANFORGE). Detailed breakdown of plan execution and state translation errors for PLANFORGE across 15 PDDL domains. The plots highlight the distribution of errors, providing insight into the model’s robustness in handling diverse planning environments.

Metric	alfworld	blocksworld	depot	ferry	floortile	frogs_jumping	goldminer	grid	grippers	hanoi	logistics	rovers	satellite	swap	visitall	Overall
Accuracy (%)	40.00	100.00	66.67	73.33	33.33	33.33	86.67	100.00	80.00	60.00	33.33	53.33	93.33	33.33	66.67	63.56

Table 9: **Domain-wise accuracy when the model is prompted to directly generate a plan.** The model is asked to output a plan without first explicitly deciding reachability. Accuracy is measured by execution-based validation, so a prediction is correct only if the generated plan is executable and reaches the goal. The model used is GPT-5.x

Domain	GT YES	Min	Max	Avg	Validated	Accuracy (%)
alfworld	32	1	10	5.6	22	68.75
blocksworld	31	2	368	93.2	31	100.00
depot	32	2	106	18.4	32	100.00
ferry	32	2	178	29.9	32	100.00
floortile	32	2	66	32.8	32	100.00
frogs_jumping	32	1	624	136.6	32	100.00
goldminer	32	2	27	13.9	30	93.75
grid	32	2	30	16.5	31	96.88
grippers	32	2	200	23.2	32	100.00
hanoi	32	1	511	53.8	11	34.38
logistics	32	2	91	27.4	32	100.00
rovers	32	2	98	27.5	32	100.00
satellite	32	2	29	10.9	31	96.88
swap	32	2	14	5.9	32	100.00
visitall	32	2	25	14.1	32	100.00
Overall	479	1	624	34.7	444	92.69

Table 10: Domain-wise plan statistics and VAL-verified accuracy on **Plan Gen**.

Domain-wise Breakdown by Plan Length Plan generation accuracy across domains remains consistently high, with failures concentrated in specific domains like hanoi (Table 11).

Accuracy vs Plan Length Plan Gen accuracy remains stable across increasing plan lengths (Table 12), achieving perfect performance beyond medium complexity while only short plans show slight degradation. If we observe the Plan Existence Figure 10 and Plan Generation Figure 2, we observe these the performance results between our approach and the existing baselines.

Key Insights

- **CoT Overestimation:** CoT predicts high plan existence but fails to generate valid executable plans, leading to near-zero VAL accuracy.
- **SCOPE Limitations:** While slightly better than CoT, SCOPE struggles with both accuracy and consistency, and incurs non-trivial cost without reliable performance gains.
- **PLANFORGE Scalability:** PLANFORGE achieves **92.69% VAL accuracy overall**, maintaining near-perfect performance across increasing plan lengths (up to 600+ steps).
- **Length is Not the Bottleneck:** Accuracy remains **100% for medium to very long plans**, indicating that failures are due to reasoning errors rather than plan complexity.
- **Failure Modes:** Errors are concentrated in specific domains (e.g., *hanoi*, *alfworld*) due to incorrect action grounding rather than inability to handle long sequences.

Conclusion These results demonstrate that **PLANFORGE is the only method that reliably scales with plan length**. Unlike CoT and SCOPE, which either overestimate feasibility or fail to generate valid

plans, PLANFORGE consistently produces correct and executable plans even under extreme planning complexity.

Final takeaway: Planning performance is not limited by plan length, but by the model’s ability to correctly construct structured, executable reasoning — a capability that PLANFORGE explicitly enforces.

Summary of Long-Horizon Scaling Takeaways The empirical results across plan length bins demonstrate that plan length is not a bottleneck for PLANFORGE, which maintains **100% VAL-verified accuracy** on all medium (11–30 steps), long (31–100 steps), and very long (100+ steps) plan categories (Table 12). By offloading search to a synthesized deterministic solver, PLANFORGE avoids autoregressive error accumulation, unlike CoT and SCOPE which degrade rapidly on long horizons. Small residual PLANFORGE errors (92.69% overall accuracy in Table 11) are confined to specific domains like *hanoi* (34.38%) due to initial state translation (NL2STATE) or macro-action formatting rather than plan length.

C.3 Token Cost, Efficiency, and Amortized Code-Reuse

Evaluating the practical viability of LLM-driven planning systems requires analyzing computational economics alongside accuracy. Existing methods rely on two distinct paradigms: per-instance online LLM inference (CoT and SCOPE), where financial and token cost scales linearly $O(N)$ with instance count N , versus amortized program synthesis (PLANFORGE), which incurs a one-time development cost during the Architect–Builder–Runner phase and achieves $O(1)$ marginal inference cost per test problem.

We report estimated LLM token costs and API call statistics incurred during automatic code generation for the Architect–Builder–Runner (ABR) framework. The estimates are obtained using the

Domain	Short	Medium	Long	Very Long	Overall
alfworld	22/32	–	–	–	68.75
blocksworld	16/16	4/4	4/4	7/7	100.00
depot	17/17	8/8	6/6	1/1	100.00
ferry	14/14	11/11	4/4	3/3	100.00
floortile	8/8	5/5	19/19	–	100.00
frogs_jumping	8/8	–	2/2	22/22	100.00
goldminer	8/10	22/22	–	–	93.75
grid	9/10	22/22	–	–	96.88
grippers	16/16	10/10	5/5	1/1	100.00
hanoi	9/30	–	–	2/2	34.38
logistics	9/9	14/14	9/9	–	100.00
rovers	9/9	12/12	11/11	–	100.00
satellite	19/20	12/12	–	–	96.88
swap	27/27	5/5	–	–	100.00
visitall	8/8	24/24	–	–	100.00

Table 11: Domain-wise **Plan Gen** accuracy across plan-length categories.

Category	Range	Total	Accuracy (%)
Short	1–10	234	85.04
Medium	11–30	149	100.00
Long	31–100	60	100.00
Very Long	100+	36	100.00

Table 12: **Plan Gen** accuracy across plan-length categories.

CodeBurn toolkit², which computes approximate usage statistics from generated execution and interaction disk log files.

- **ACPBench** – multi-domain classical planning benchmark consisting of 15 planning domains. Which includes both **Plan Gen** and **Plan Gen Easy** costs across these 2 datasets with comparison with baselines, referring to Table 13 for **Plan Gen Easy** and Table 15 for **Plan Gen**.
- **NaturalPlan** – natural-language planning benchmark containing Trip Planning, Meeting Planning, and Calendar Scheduling tasks, referring to Table 16
- **StructureSAT** – SAT reasoning benchmark consisting of 9 structurally diverse SAT problem families, where the objective is to determine satisfiability and produce valid satisfying assignments when solutions exist, referring to Table 17

In this section, we report the estimated cost analysis for ACPBench, NaturalPlan, and StructureSAT under PLANFORGE and compare it against SCOPE and CoT (see the Figure 11) . Since PLANFORGE is a domain-specific Architect–Builder–Runner framework, the reported PLANFORGE costs include both development and evaluation phases and are measured over substantially larger evaluation workloads. For NaturalPlan and StructureSAT, PLANFORGE is evaluated on at least 1,000 instances per sub-domain after the development phase. In contrast, the reported CoT and SCOPE cost statistics for NaturalPlan and StructureSAT are computed using only 150 instances per sub-domain. For Plan Gen Easy, all methods are evaluated under the same setting, where each of the 15 ACPBench domains contains 15 benchmark instances. And for **Plan Gen**, there are 45 instances in each of these 15 domains. Therefore, the reported cost statistics

should be interpreted with these differences in evaluation scale and methodology in mind.

Cost-Efficiency Takeaways While PLANFORGE incurs an initial offline development cost (e.g., $\sim$ \$17.24 across 15 ACPBench domains), its marginal inference cost per test instance is \$0.00. On large-scale workloads of 1,000+ instances per domain (NaturalPlan and StructureSAT), PLANFORGE yields up to a $10\times$ – $50\times$ **reduction in total token cost** compared to per-instance search baselines while simultaneously delivering superior planning accuracy.

D Extended Related Work

We position PLANFORGE in the context of programmatic and neurosymbolic planning. A detailed survey of classical planning grounding, LLM agents, and search-based heuristics is provided in Appendix D.

Programmatic and Neurosymbolic Planning. Recent neurosymbolic frameworks divide labor between neural perception (converting raw inputs to structures) and symbolic reasoning (executing search or checking constraints) (Sheth, Roy, and Gaur 2023). Several approaches translate natural language into formal planning specifications (e.g., PDDL) to run classical solvers (Liu et al. 2023), or compile problems into programmatic workflows that generate and filter candidates. For eg., SCOPE (Deik et al. 2026). However, these frameworks rely on predefined, human-designed planner blueprints (representations, function signatures, or fixed algorithmic templates). This assumption limits their applicability to novel planning problems described in natural language. In contrast, PLANFORGE automates the end-to-end process of synthesizing planners by dynamically inducing the blueprint.

²<https://github.com/getagentseal/codeburn>

Domain	CoT	SCOPE	Guided	PLANFORGE
alfworld	0.14 / 15	0.22 / 25	2.72 / 223	0.97 / 25
blocksworld	0.08 / 15	0.22 / 28	0.09 / 18	1.77 / 32
depot	0.09 / 15	0.17 / 26	1.55 / 121	0.93 / 15
ferry	0.05 / 15	0.18 / 24	0.17 / 23	0.93 / 15
floortile	0.10 / 15	0.42 / 26	0.24 / 25	1.03 / 21
frogs_jumping	0.08 / 15	0.38 / 24	0.20 / 21	0.75 / 16
goldminer	0.07 / 15	0.27 / 24	1.62 / 139	1.19 / 22
grid	0.05 / 15	0.23 / 24	0.07 / 11	0.71 / 15
grippers	0.06 / 15	0.13 / 25	0.39 / 45	1.07 / 21
hanoi	0.10 / 15	0.37 / 25	0.16 / 15	1.47 / 23
logistics	0.07 / 15	0.21 / 26	0.79 / 55	1.77 / 29
rovers	0.13 / 15	0.32 / 25	0.35 / 30	1.71 / 22
satellite	0.10 / 15	0.19 / 26	2.00 / 133	1.20 / 17
swap	0.05 / 15	0.19 / 27	0.16 / 24	0.92 / 11
visitall	0.06 / 15	0.16 / 24	1.13 / 97	0.83 / 17
Total	1.23 / 225	3.65 / 379	11.62 / 980	17.24 / 301

Table 13: Cost (\$) / LLM calls comparison on **Plan Gen Easy**. Each cell shows “cost / calls”. PLANFORGE refers to the Autonomous ABR variant used in the main results.

Search, Heuristics, and Generalized Planning. Interactive LMs generate plans via step-by-step action text, but their outputs are prone to execution errors and unreachability hallucination (Valmeekam et al. 2023b; Kambhampati et al. 2024; Bohnet et al. 2026). To enforce correctness, recent methods prompt LMs to generate search components or heuristic functions (Katz et al. 2024; Corrêa, Pereira, and Seipp 2025; Cao et al. 2026). Other generalized planning works synthesize reusable programs but assume expert-designed schemas (Silver et al. 2024; Chen et al. 2025). PLANFORGE resolves this bottleneck by synthesizing both the search strategy and the underlying representation autonomously.

Domain	CoT	SCOPE	PLANFORGE
alfworld	26.70	40.00	100.00
blocksworld	26.70	40.00	100.00
depot	26.70	40.00	100.00
ferry	33.30	33.30	100.00
floortile	26.70	26.70	100.00
frogs_jumping	26.70	66.70	100.00
goldminer	20.00	46.70	100.00
grid	33.30	66.70	100.00
grippers	33.30	26.70	100.00
hanoi	13.30	40.00	100.00
logistics	20.00	6.70	100.00
rovers	20.00	40.00	100.00
satellite	33.30	40.00	100.00
swap	33.30	40.00	100.00
visitall	33.30	80.00	100.00
Plan Gen (Overall)	27.10	42.20	100.00

Table 14: Comparison of **Plan Generation Accuracy (%)** for CoT, SCOPE, and **PLANFORGE** on **Plan Gen Easy** across 15 planning domains.

Domain	CoT VAL Acc (%)	SCOPE VAL Acc (%)	PLANFORGE VAL Acc (%)	CoT Cost (\$)	SCOPE Cost (\$)	PLANFORGE Cost (\$)
alfworld	0.00	0.00	68.75	0.1316	0.2198	1.08
blocksworld	0.00	3.12	100.00	0.3485	0.2249	1.11
depot	0.00	3.12	100.00	0.1712	0.1720	1.06
ferry	0.00	3.12	100.00	0.1863	0.1787	1.37
floortile	0.00	6.25	100.00	0.4016	0.4174	1.23
frogs_jumping	0.00	0.00	100.00	0.3456	0.3773	1.30
goldminer	0.00	3.12	93.75	0.1217	0.2665	1.10
grid	0.00	15.62	96.88	0.1763	0.2261	1.18
grippers	0.00	3.12	100.00	0.1835	0.1312	1.03
hanoi	0.00	3.12	34.38	0.8936	0.3745	1.34
logistics	3.12	3.12	100.00	0.2820	0.2103	1.18
rovers	0.00	3.12	100.00	0.2503	0.3181	0.86
satellite	6.25	3.12	96.88	0.1667	0.1889	1.54
swap	0.00	3.12	100.00	0.1459	0.1891	0.67
visitall	0.00	18.75	100.00	0.2223	0.1645	0.72
Plan Gen (Overall)	0.63	4.80	92.69	4.0271	3.6595	16.77

Table 15: Domain-wise VAL-verified plan-generation accuracy and API cost on **Plan Gen**, our extended long-horizon ACPBench evaluation with 36 reachable instances per domain.

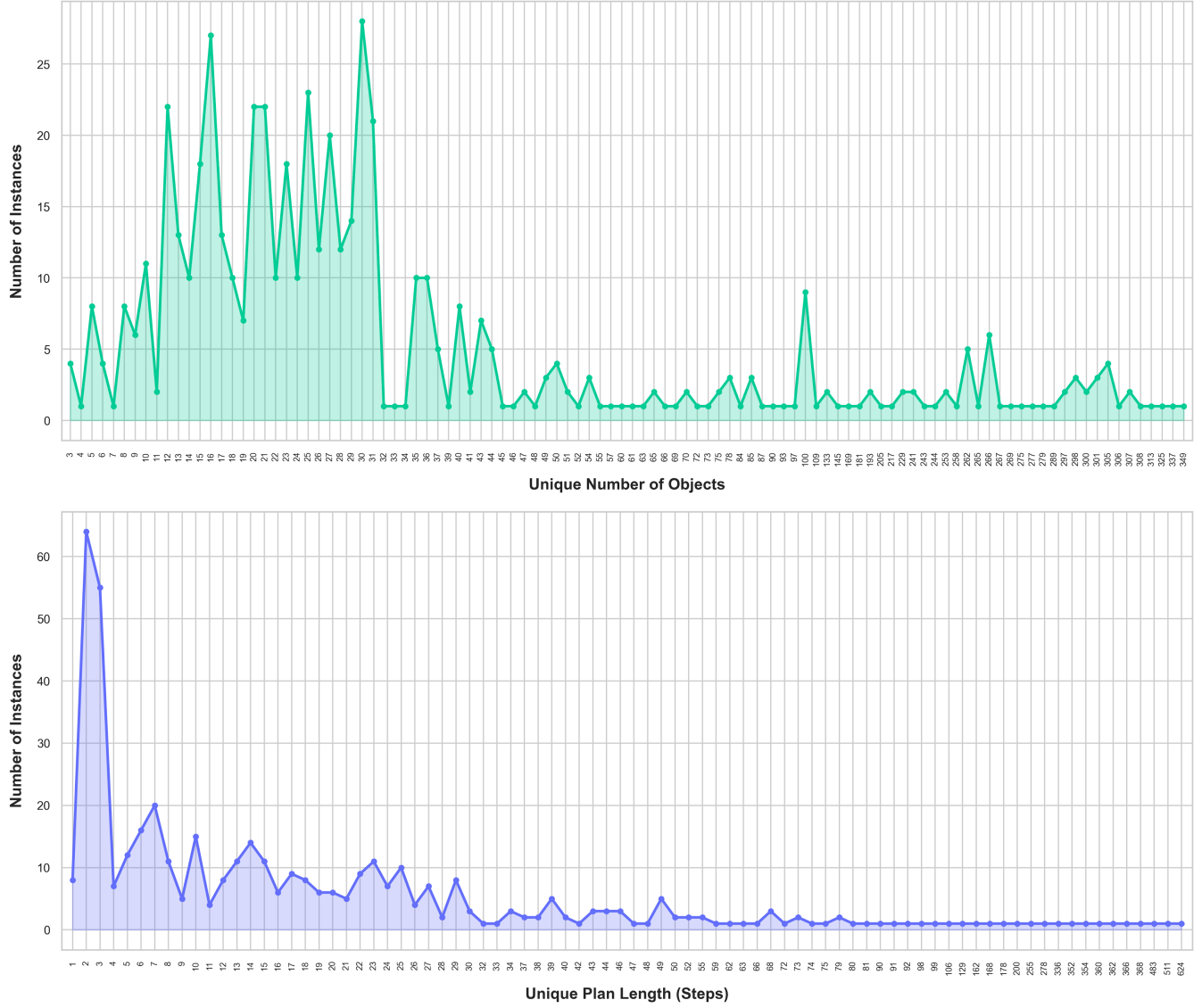


Figure 9: **Plan Gen dataset statistics.** (a) Distribution of plan lengths across all instances. (b) Distribution of problem complexity measured by number of objects. The dataset spans a wide range of planning difficulty, including very long plans (up to 600+ steps).

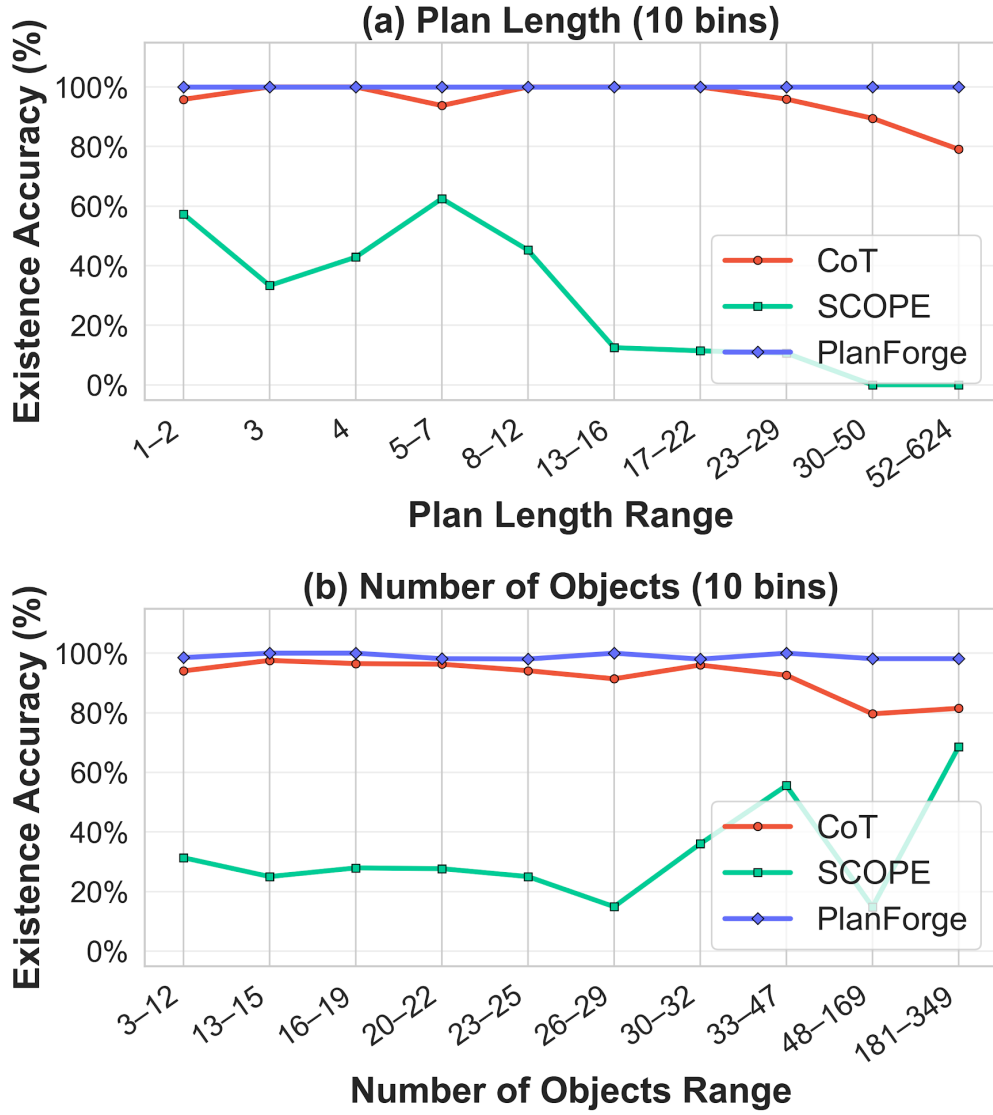


Figure 10: **Plan Gen plan-existence accuracy.** (a) Across plan length ranges. (b) Across number of objects. CoT tends to overestimate plan feasibility, while SCOPE shows unstable performance. PLANFORGE remains consistently accurate.

Method	Trip	Meeting	Calendar	Total
<i>Cost (\$)</i>				
CoT	0.68	0.63	0.58	1.89
SCOPE	0.83	1.26	1.36	3.44
PLANFORGE-Guided	0.78	1.01	1.76	3.55
PLANFORGE (Ours)	1.06	0.68	1.47	3.21
<i>LLM Calls</i>				
CoT	150	150	150	450
SCOPE	159	174	184	517
PLANFORGE-Guided	15	18	36	69
PLANFORGE (Ours)	23	16	25	64

Table 16: Cost and LLM call comparison on NaturalPlan domains. PLANFORGE refers to the Autonomous ABR variant used in the main results.

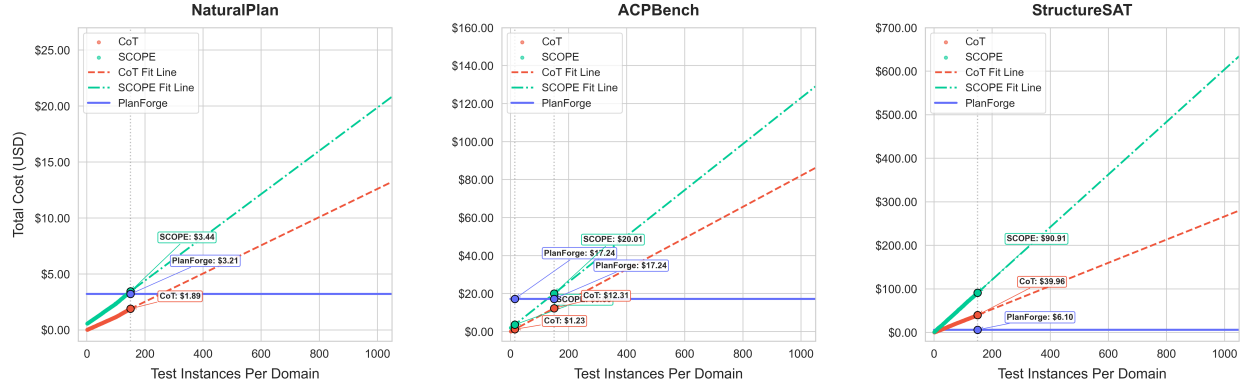


Figure 11: Cumulative API cost (USD) versus the number of evaluated test instances, including development and test-time costs. **PLANFORGE** incurs a one-time development cost and requires no test-time API queries, so its total cost remains constant. In contrast, **CoT** and **SCOPE** query the model for each instance, causing their costs to increase with evaluation size.

Domain	CoT		SCOPE-Based Implementation		PLANFORGE	
	Cost (\$)	LLM Calls	Cost (\$)	LLM Calls	Cost (\$)	LLM Calls
3-sat	3.42	149	3.91	156	0.37	8
automor	6.94	150	27.71	157	0.90	18
ca	5.57	150	16.98	150	0.59	8
k-clique	4.55	149	10.45	150	0.95	11
k-color	4.89	148	2.30	150	0.51	11
sr	2.94	150	7.16	157	0.63	11
k-domset	4.68	150	7.51	156	0.69	10
k-vercov	4.56	150	7.01	156	0.72	7
ps	2.41	150	7.88	156	0.73	10
Total	39.96	1346	90.91	1388	6.10	94

Table 17: Estimated LLM cost and API call statistics for **CoT**, **SCOPE**, and **PLANFORGE** across StructureSAT domains. Costs are estimated using CodeBurn from execution disk logs.